

# 3\_Python\_clusteringAnnotation

V. R. Knight-Schrijver

September 3, 2026

```
[1]: import os
import tempfile
import time
import re
import torch
import scanpy as sc
import scvi
import pandas as pd
import gc
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

/home/vincent/miniconda3/envs/scvi-env/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See [https://ipywidgets.readthedocs.io/en/stable/user\\_install.html](https://ipywidgets.readthedocs.io/en/stable/user_install.html)  
from .autonotebook import tqdm as notebook\_tqdm

## 1 Annotation and Clustering

### 1.0.1 Load in data

```
[2]: # Read in the model-trained object
bdata = sc.read_h5ad("data/suspension/anndata_integration_temp.h5ad")
```

```
[3]: # Read in the combined whole object
adata = sc.read_h5ad("data/suspension/anndata_combined.h5ad")
```

```
[5]: adata = adata[bdata.obs_names]
```

```
[6]: # transfer calculated embeddings etc to ADATA
adata.obsm["X_scVI"] = bdata.obsm["X_scVI"]
adata.obsm["X_umap"] = bdata.obsm["X_umap"]
adata.obs["leiden_1.5"] = bdata.obs["leiden_1.5"]
adata.obs["leiden_1"] = bdata.obs["leiden_1"]
adata.obs["leiden_0.5"] = bdata.obs["leiden_0.5"]
adata.obs["leiden_0.2"] = bdata.obs["leiden_0.2"]
```

/tmp/ipykernel\_52419/1328103092.py:2: ImplicitModificationWarning: Setting element `obsm['X\_scVI']` of view, initializing view as actual.

```
adata.obsm["X_scVI"] = bdata.obsm["X_scVI"]
```

```
[7]: # Saving count data
adata.layers["counts"] = adata.X.copy()
```

```
[8]: adata.layers["counts"] = np.round(adata.layers["counts"])
```

```
[9]: # Normalizing to median total counts
sc.pp.normalize_total(adata)
# Logarithmize the data
sc.pp.log1p(adata)

[10]: sc.tl.dendrogram(adata, "leiden_1.5", n_pcs=None, use_rep="X_scVI")

[11]: marker_genes = {
    "vCMs": ["MYH7", "TNNT2"],
    "aCMs": ["MYH6", "NR2F2"],
    "CCS": ["VSNL1", "HCN4"],
    "cSMCs": ["MYH11", "ACTA2"],
    "cPericytes": ["RGS5", "CYGB"],
    "Endocardial": ["NPR3", "ADGRG6", "PECAM1"],
    "Artery": ["HEY1", "FBLN5"],
    "Capillary": ["RGCC", ],
    "Venous": ["ACKR1", "NR2F2"],
    "Lymphatic_Endo": ["TFF3"],
    "Valves": ["NFATC1"],
    "Epicardial": ["ITLN1", "UPK3B"],
    "Fibroblasts_etc": ["LUM", "COL1A1"],
    "ImmuneCells": ["PTPRC"],
    "SchwannCells": ["MPZ", "S100B"],
    "Neurons": ["NRXN1", "ENO2"],
    "Chromaffin": ["CHGA", "CHGB"],
    "Megakaryocyte": ["ITGB3", "ITGA2B"],
    "Adipocytes": ["ADIPOQ"],
    "RBCs": ["HBB"],
    "EDNs": ["EDNRA", "EDNRB", "EDN1", "EDN2", "EDN3", "ECE1", "ECE2"]
}
```

## 1.1 1. Annotation using manual merge of previous labels

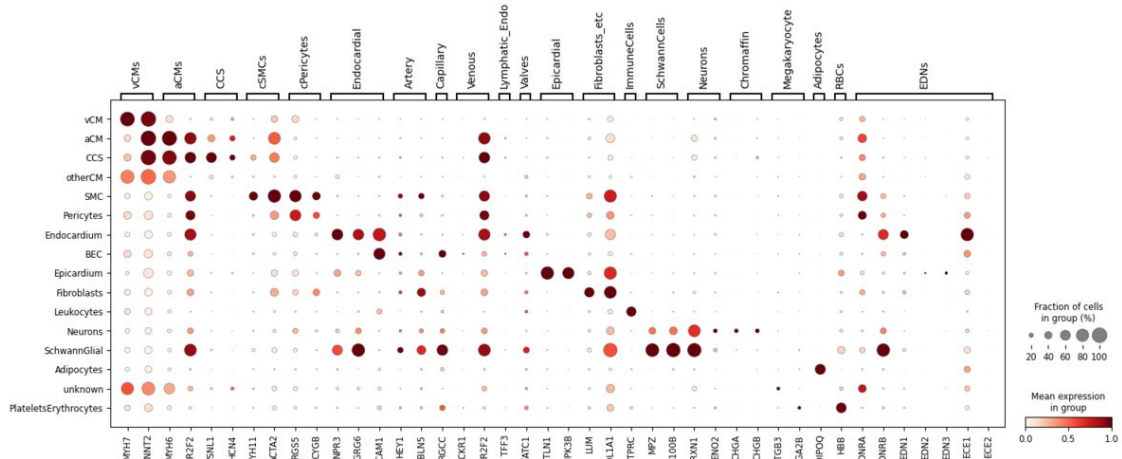
```
[13]: labels_vCM = ["Immat_CM", "Immature_Cardiomyocytes", "LV1", "LV2", "LV3", "LV4", "LV5",
    "LV6", "LV7", "Mat_vCM", "MetAct_vCM_1", "MetAct_vCM_2", "Prol_CM", "vCM"]
labels_aCM = ["LA1", "LA2", "LA3", "LA4", "LA5", "LA6", "Mat_aCM", "aCM", "MetAct_aCM"]
labels_CCS = ["AV", "ncCM"]
labels_otherCM = ["cardiac muscle myoblast", "Cardiomyocytes"]
labels_cSMC = ["CA_SMC", "OFT_SMC", "SMC", "SMC1", "SMC2", "SMC3", "SMC4",
    "SMC5", "Smooth_Muscle_Cells", "TMSB10high_C_1", "smooth muscle myoblast"]
labels_cPericytes = ["PC", "Peric_MC", "Pericytes", "Pericytes_Stromal", "pericyte"]
labels_Endocardium = ["EndocCush_EC", "Endoc_EC", "Endocardial", ""]
labels_BEC = ["BEC", "EC1", "EC2", "EC3", "EC4", "EC5", "EC6", "EC7", "Endothelial_Other",
    "MacroVasc_EC", "MicroVasc_EC", "PDE4Chigh_EC", "TMSB10high_C_2", "cardiac endothelial cell",
    ↪ "Endothelial_Venous", "Endothelial_Arterial", "Endothelial_Capillaries", "LEC"]
#labels_EndothelialArteries = ["Endothelial_Arterial"]
#labels_EndothelialCapillaries = ["Endothelial_Capillaries"]
#labels_EndothelialVenous = ["Endothelial_Venous"]
#labels_EndothelialLymphatic = ["LEC"]
labels_Epicardium = ["EpC", "Epicardial", "Epicardium_FB-like", "Epicardium_Meso", "Epicardium_Proliferating"]
labels_Fibroblasts = ["AnnFibr_FB", "FB1", "FB2", "FB3", "FB4", "FB5", "FB6",
    "Fibro", "Fibroblasts", "Int_FB", "OFT_FB", "PDE4Chigh_FB", "Prol_FB", "Valve_MC", "fibroblast of cardiac_
    ↪ tissue", "EPDC"]
labels_Leukocytes =
    ↪ ["Lymphoid_Immune_Cells", "MP1", "MP2", "MP3", "MP4", "MyC", "LyC", "Myeloid_Immune_Cells", "WBC", "immature_
    ↪ innate lymphoid cell", "lymphoid lineage restricted progenitor cell", "mast cell"]
labels_Neurons = ["NB-N", "Neuronal", "Neuronal_Cells", "neuronal receptor cell"]
labels_SchwannGlial = ["SCP-GC"]
labels_Adipocytes = ["Adipocytes", "adipocyte of epicardial fat of left ventricle"]
labels_unknown = ["HL_excl_2", "HL_excl_3", "Immature_other", "unknown", "unlabelled"]
labels_PlateletsErythrocytes = ["P-RBC", "HL_excl_1", "HL_excl_4"]

[15]: adata.obs["merged_labels"] = pd.Series("unknown", dtype="category")
```

```
[16]: adata.obs["merged_labels"] = adata.obs["merged_labels"].cat.set_categories(["vCM", "aCM", "CCS", "otherCM",
↳ "SMC", "Pericytes", "Endocardium", "BEC", "Epicardium", "Fibroblasts", "Leukocytes", "Neurons",
↳ "SchwannGlial", "Adipocytes", "unknown", "PlateletsErythrocytes"])
```

```
[17]: adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_vCM)] = "vCM"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_aCM)] = "aCM"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_CCS)] = "CCS"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_otherCM)] = "otherCM"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_cSMC)] = "SMC"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_cPericytes)] = "Pericytes"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_Endocardium)] = "Endocardium"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_BEC)] = "BEC"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_Epicardium)] = "Epicardium"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_Fibroblasts)] = "Fibroblasts"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_Leukocytes)] = "Leukocytes"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_Neurons)] = "Neurons"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_SchwannGlial)] = "SchwannGlial"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_Adipocytes)] = "Adipocytes"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_unknown)] = "unknown"
adata.obs["merged_labels"][np.isin(adata.obs["cell_type_mid"], labels_PlateletsErythrocytes)] =
↳ "PlateletsErythrocytes"
```

```
[18]: sc.pl.dotplot(adata, marker_genes, groupby="merged_labels", standard_scale="var")
```



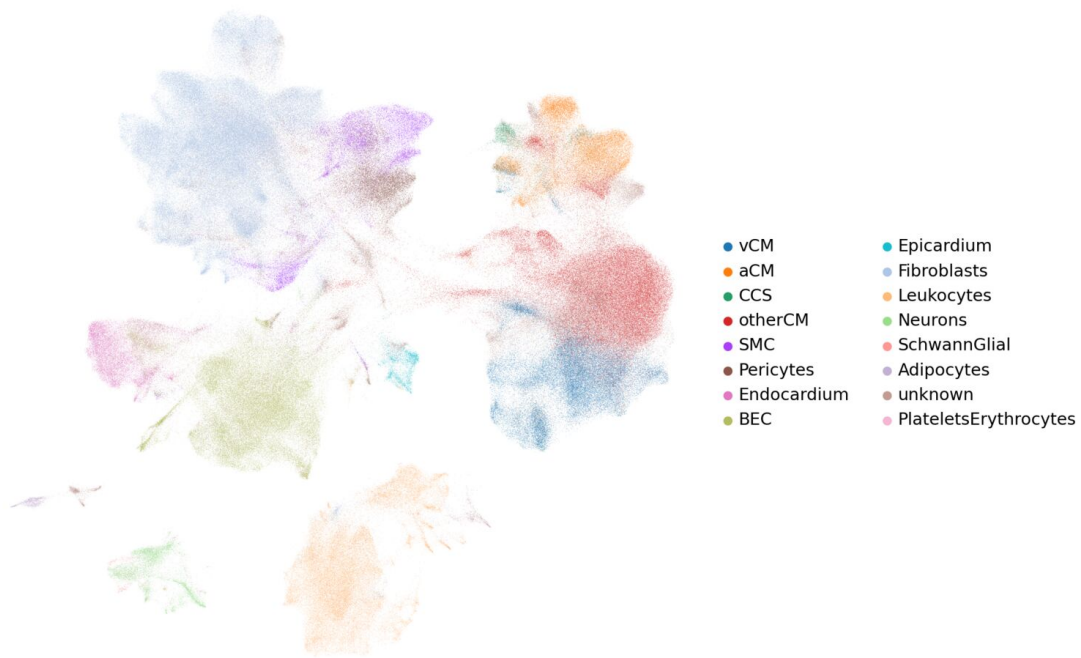
```
[19]: # check for NA factor levels
adata.obs["cell_type_mid"][pd.isna(adata.obs["merged_labels"])]
```

```
[19]: Series([], Name: cell_type_mid, dtype: category
Categories (112, object): ['AV', 'Adipocytes', 'AnnFibr_FB', 'BEC', ..., 'smooth
muscle myoblast', 'unknown', 'unlabelled', 'vCM'])
```

```
[20]: # add to bdata
bdata.obs["merged_labels"] = adata.obs["merged_labels"]
```

```
[21]: sc.set_figure_params(figsize=(10, 10), frameon=False)
sc.pl.umap(
    bdata,
    color=["merged_labels"]
    #legend_loc = "on data",
    #ncols=2
)
```

merged\_labels

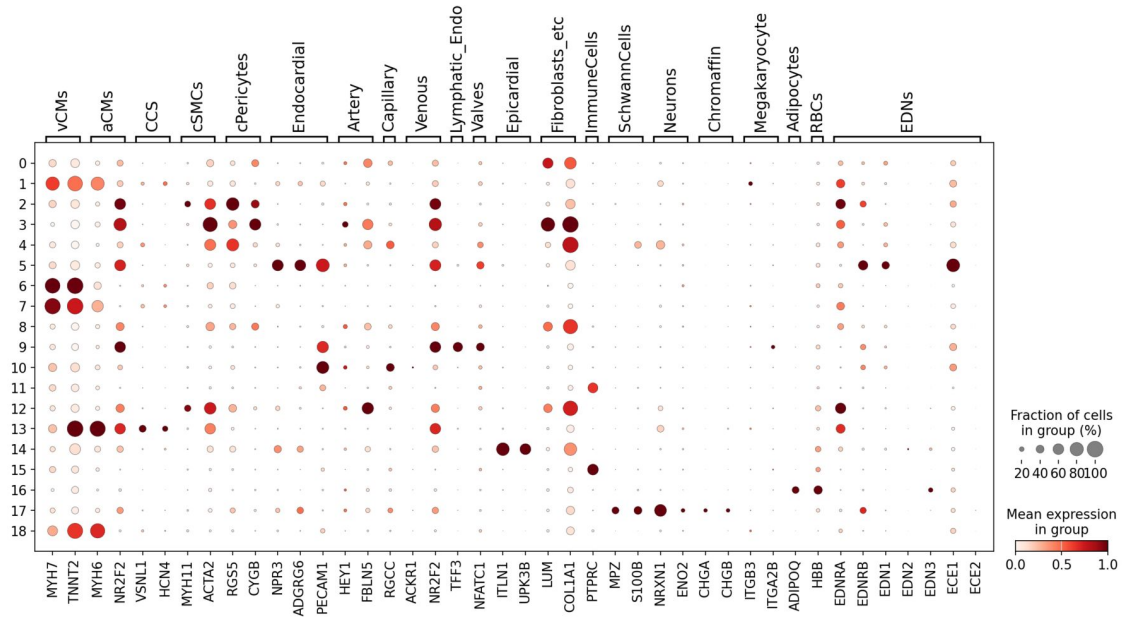


```
[22]: # send original annotations dataset matrix to csv
pd.crosstab(adata.obs["dataset"], adata.obs["cell_type_mid"]).to_csv("dataset_cellType_matrix.csv")
# send updated annotations merged_cell type matrix to csv
pd.crosstab(adata.obs["cell_type_mid"], adata.obs["merged_labels"]).to_csv("merged_labels_cellType_matrix.
↳ csv")
```

```
[23]: # plot marker genes as dotplot
sc.pl.dotplot(adata, marker_genes, groupby="leiden_0.5", standard_scale="var", save="fuzzyDotPlot.pdf")
```

WARNING: saving figure to file figures/dotplot\_fuzzyDotPlot.pdf





1.1.1 Clearly there is still a bit of fuzziness in the clustering... We may be able to re-combine the labels and re-cluster:

1. Cardiomyocytes
2. Endothelial Cells (Lymphatics / BloodVessel types)
3. Neurons

### 1.1.2 Cardiomyocyte subclustering

```
[24]: # combined dataset now
#adata = sc.read_h5ad("data/suspension/anndata_combined.h5ad")
```

```
[25]: #adata.obs["clusters_res_low"] = bdata.obs["clusters_res_low"]
```

```
[26]: # subset object(s) for further processing
# Cardiomyocytes only
CM_adata = adata[np.isin(adata.obs["merged_labels"], ["vCM", "aCM", "CCS", "otherCM"])]
```

```
[27]: # casting as CSR matrix
CM_adata.layers["counts"] = CM_adata.layers["counts"].tocsr()
```

```
[28]: sc.pp.highly_variable_genes(
    CM_adata,
    n_top_genes=4000,
    subset=True,
    layer="counts",
    flavor="seurat_v3",
    batch_key="dataset"
)
```

```
[29]: CM_adata = CM_adata.copy()
```

```
[30]: variables2merge = ["stage", "cell_or_nuclei", "donor", "tech", "disease_condition"]
```

```
[31]: scvi.model.SCVI.setup_anndata(
    CM_adata,
    layer="counts",
    categorical_covariate_keys=variables2merge,
    continuous_covariate_keys=None,
)
```

```
[32]: CM_model = scvi.model.SCVI(
    CM_adata,
    n_layers=2,
    n_hidden=128,
    n_latent=10,
    gene_likelihood="nb"
)
```

```
[33]: dataspitter_kwargs = {}
dataspitter_kwargs["drop_dataset_tail"] = True
dataspitter_kwargs["drop_last"] = False
```

```
[34]: CM_model.train(
    max_epochs=40,
    check_val_every_n_epoch=1,
    accelerator="gpu",
    devices=1
    #dataspitter_kwargs=dataspitter_kwargs,
    #strategy="ddp_notebook_find_unused_parameters_true"
)
```

GPU available: True (cuda), used: True

TPU available: False, using: 0 TPU cores

HPU available: False, using: 0 HPUs

LOCAL\_RANK: 0 - CUDA\_VISIBLE\_DEVICES: [0,1]

/home/vincent/miniconda3/envs/scvi-env/lib/python3.13/site-

packages/lightning/pytorch/trainer/connectors/data\_connector.py:433: The  
'train\_dataloader' does not have many workers which may be a bottleneck.

Consider increasing the value of the `num\_workers` argument` to `num\_workers=15`  
in the `DataLoader` to improve performance.

/home/vincent/miniconda3/envs/scvi-env/lib/python3.13/site-

packages/lightning/pytorch/trainer/connectors/data\_connector.py:433: The  
'val\_dataloader' does not have many workers which may be a bottleneck. Consider  
increasing the value of the `num\_workers` argument` to `num\_workers=15` in the  
`DataLoader` to improve performance.

Epoch 40/40: 100%|| 40/40 [06:44<00:00, 10.18s/it, v\_num=1,  
train\_loss=1.84e+3]

`Trainer.fit` stopped: `max\_epochs=40` reached.

Epoch 40/40: 100%|| 40/40 [06:44<00:00, 10.11s/it, v\_num=1,  
train\_loss=1.84e+3]

```
[35]: # save model
CM_model.save("data/suspension/scvi_model_CM", overwrite=True)
#CM_model = scvi.model.SCVI.load("data/suspension/scvi_model_CM", adata=CM_adata)
```

```
[36]: # add latent space to object
SCVI_LATENT_KEY = "X_scVI"
latent = CM_model.get_latent_representation()
CM_adata.obsm[SCVI_LATENT_KEY] = latent
```

```

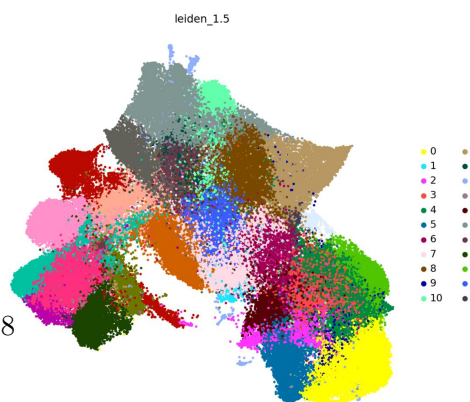
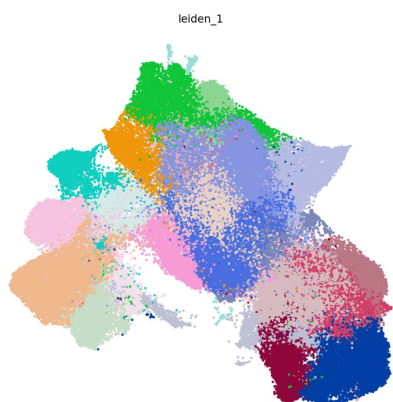
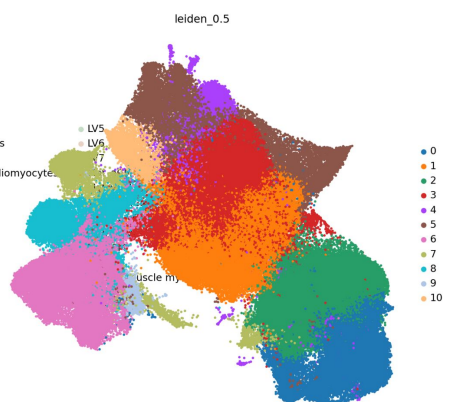
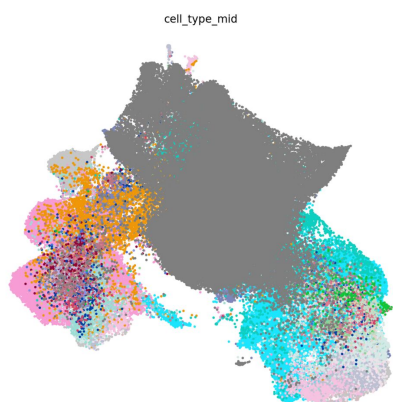
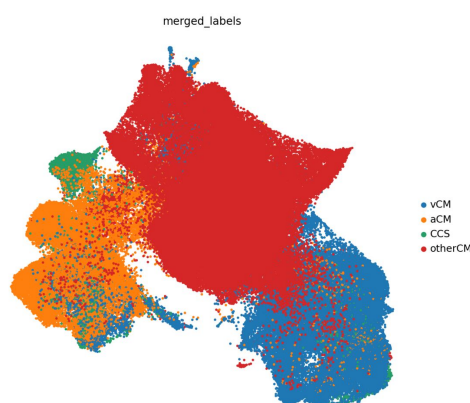
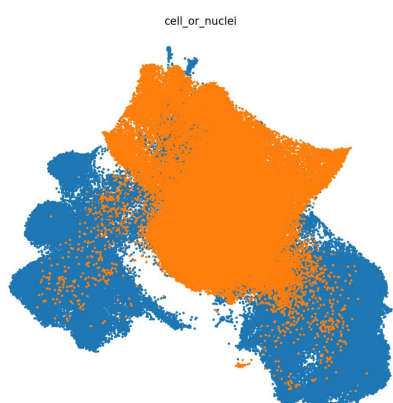
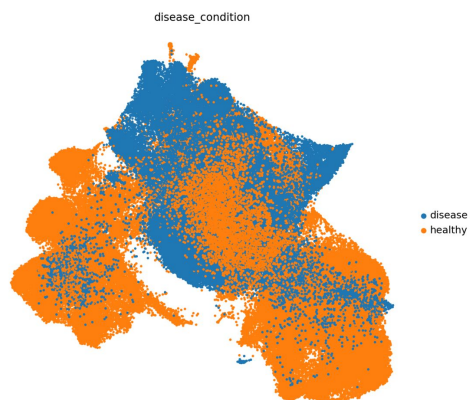
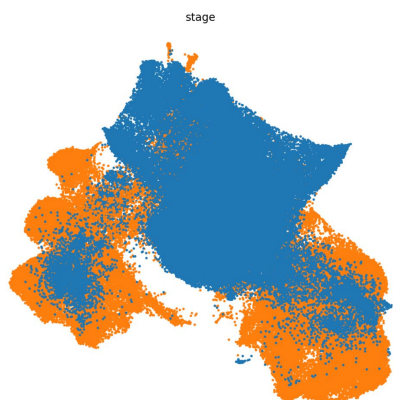
[37]: # use scVI latent space for UMAP generation
sc.pp.neighbors(CM_adata, use_rep="X_scVI", n_neighbors = 15)
sc.tl.umap(CM_adata, min_dist=0.3)

[38]: sc.tl.leiden(CM_adata, flavor="igraph", resolution = 1.5, key_added='leiden_1.5')
sc.tl.leiden(CM_adata, flavor="igraph", resolution = 1, key_added='leiden_1')
sc.tl.leiden(CM_adata, flavor="igraph", resolution = 0.2, key_added='leiden_0.2')
sc.tl.leiden(CM_adata, flavor="igraph", resolution = 0.5, key_added='leiden_0.5')

[39]: sc.set_figure_params(figsize=(10, 10), frameon=False)
sc.pl.umap(
    CM_adata,
    color=["stage", "disease_condition", "cell_or_nuclei", "merged_labels", "cell_type_mid", "leiden_0.5",
↪ "leiden_1", "leiden_1.5"],
    ncols=2,
    #legend_loc = "on data",
    s=40,
    save="_Cardiomyocyte_clustering.pdf"
)

```

WARNING: saving figure to file figures/umap\_Cardiomyocyte\_clustering.pdf



```
[40]: pd.crosstab(CM_adata.obs["leiden_1"], CM_adata.obs["stage"])
```

```
[40]: stage      adult  fetal
leiden_1
0           308   9622
1           1939    674
2            148   3343
3           1938  14156
4            148   6471
5             67   4555
6          15249    442
7          13181    938
8           4788    348
9           1259   3450
10            335     14
11           1585    510
12           5220    284
13           3175    201
14            282   3893
15          11407    617
16          2079  11294
17          2878     76
18           273   2671
19             9    289
20          2637   1533
21             12   1816
22            152  6561
23          5943    193
```

```
[41]: pd.crosstab(CM_adata.obs["leiden_1"], CM_adata.obs["cell_type_mid"]).
      ↳to_csv("Cardiomyocyte_subset_leiden1p0_reannotationMatrix.csv")
pd.crosstab(CM_adata.obs["leiden_1"], CM_adata.obs["cell_type_mid"])
```

```
[41]: cell_type_mid  AV  Cardiomyocytes  Immat_CM  Immature_Cardiomyocytes  LA1  \
leiden_1
0           27         139        1142             742      0
1            0         638           0              4      0
2            2         443           2             10      2
3           32        1166          20              6     10
4            2         274           0              5      4
5            0          99          32            137      0
6            0         997           4              3      0
7            6         121           1             10      6
8            0          87           0             21      0
9           30         455          81             56     16
10           0           0           0              1      0
11          56          38           9              7     54
12           7          22           3             28      1
13           0         114           5              6      0
14          110          75          132             78     17
15           0         154           0              0      0
16          303         256           1              3    383
17           8         235           1              4      2
18          11         134           1              8    140
19           0           4          77              9      0
20          19        2449           0             22      2
21           0           3           2             36      0
22          23         134           0              5     11
23           2         115           0              0      3

cell_type_mid  LA2  LA3  LA4  LA5  LA6  ...  Mat_aCM  Mat_vCM  MetAct_aCM  \
leiden_1
0            0     1     1     0     0  ...      11      68      330
```

|    |     |     |     |     |    |     |      |      |      |
|----|-----|-----|-----|-----|----|-----|------|------|------|
| 1  | 0   | 2   | 0   | 0   | 0  | ... | 6    | 499  | 1    |
| 2  | 0   | 1   | 0   | 0   | 0  | ... | 11   | 203  | 21   |
| 3  | 0   | 13  | 3   | 2   | 0  | ... | 7    | 496  | 3    |
| 4  | 0   | 0   | 0   | 0   | 0  | ... | 1    | 434  | 3    |
| 5  | 0   | 0   | 0   | 0   | 0  | ... | 7    | 126  | 20   |
| 6  | 0   | 1   | 0   | 0   | 0  | ... | 40   | 118  | 2    |
| 7  | 2   | 3   | 0   | 1   | 0  | ... | 147  | 623  | 7    |
| 8  | 0   | 1   | 0   | 0   | 0  | ... | 79   | 86   | 1    |
| 9  | 4   | 8   | 2   | 0   | 1  | ... | 174  | 2849 | 88   |
| 10 | 0   | 0   | 0   | 0   | 0  | ... | 3    | 0    | 2    |
| 11 | 6   | 58  | 7   | 4   | 1  | ... | 5    | 86   | 19   |
| 12 | 0   | 1   | 0   | 0   | 0  | ... | 50   | 54   | 47   |
| 13 | 0   | 0   | 0   | 0   | 0  | ... | 41   | 116  | 3    |
| 14 | 8   | 2   | 4   | 4   | 3  | ... | 106  | 6    | 2083 |
| 15 | 0   | 0   | 1   | 0   | 0  | ... | 64   | 359  | 1    |
| 16 | 223 | 121 | 138 | 178 | 26 | ... | 667  | 5    | 240  |
| 17 | 0   | 0   | 0   | 0   | 0  | ... | 21   | 10   | 4    |
| 18 | 8   | 37  | 1   | 1   | 0  | ... | 286  | 11   | 34   |
| 19 | 0   | 0   | 0   | 0   | 1  | ... | 21   | 17   | 2    |
| 20 | 1   | 8   | 1   | 0   | 0  | ... | 1151 | 10   | 48   |
| 21 | 0   | 0   | 0   | 0   | 0  | ... | 91   | 4    | 42   |
| 22 | 6   | 11  | 0   | 4   | 0  | ... | 672  | 1    | 58   |
| 23 | 5   | 7   | 1   | 0   | 0  | ... | 96   | 12   | 1    |

| cell_type_mid<br>leiden_1 | MetAct_vCM_1 | MetAct_vCM_2 | Prol_CM | aCM \ |
|---------------------------|--------------|--------------|---------|-------|
| 0                         | 2315         | 1105         | 985     | 5     |
| 1                         | 4            | 1            | 0       | 0     |
| 2                         | 27           | 0            | 1       | 32    |
| 3                         | 192          | 61           | 0       | 1     |
| 4                         | 67           | 3            | 2       | 0     |
| 5                         | 95           | 13           | 174     | 1     |
| 6                         | 14           | 4            | 0       | 27    |
| 7                         | 6            | 1            | 0       | 14    |
| 8                         | 2            | 0            | 8       | 10    |
| 9                         | 62           | 17           | 35      | 3     |
| 10                        | 1            | 4            | 0       | 2     |
| 11                        | 47           | 15           | 0       | 0     |
| 12                        | 13           | 2            | 24      | 26    |
| 13                        | 7            | 1            | 6       | 2     |
| 14                        | 365          | 189          | 281     | 393   |
| 15                        | 1            | 1            | 0       | 34    |
| 16                        | 7            | 0            | 1       | 10161 |
| 17                        | 0            | 0            | 0       | 20    |
| 18                        | 2            | 2            | 1       | 169   |
| 19                        | 2            | 13           | 123     | 9     |
| 20                        | 1            | 2            | 0       | 173   |
| 21                        | 2            | 0            | 52      | 1566  |
| 22                        | 0            | 0            | 0       | 5729  |
| 23                        | 1            | 0            | 0       | 67    |

| cell_type_mid<br>leiden_1 | cardiac | muscle | myoblast | ncCM | vCM   |
|---------------------------|---------|--------|----------|------|-------|
| 0                         |         |        | 78       | 221  | 2591  |
| 1                         |         |        | 1402     | 1    | 53    |
| 2                         |         |        | 88       | 61   | 2559  |
| 3                         |         |        | 960      | 6    | 12618 |
| 4                         |         |        | 10       | 2    | 5684  |
| 5                         |         |        | 63       | 6    | 3848  |
| 6                         |         |        | 14278    | 0    | 203   |
| 7                         |         |        | 13064    | 0    | 103   |
| 8                         |         |        | 4718     | 0    | 123   |
| 9                         |         |        | 702      | 4    | 24    |
| 10                        |         |        | 335      | 1    | 0     |
| 11                        |         |        | 278      | 0    | 301   |
| 12                        |         |        | 5189     | 5    | 28    |
| 13                        |         |        | 3061     | 2    | 8     |

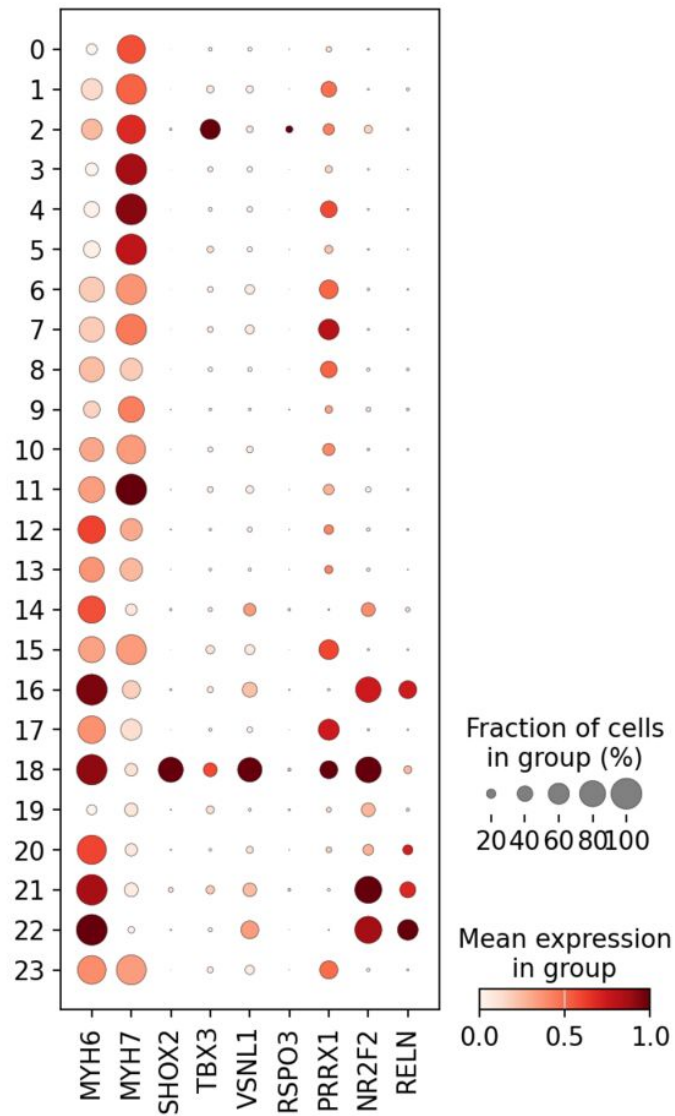
|    |  |       |      |     |
|----|--|-------|------|-----|
| 14 |  | 9     | 225  | 8   |
| 15 |  | 11282 | 0    | 126 |
| 16 |  | 190   | 30   | 2   |
| 17 |  | 2630  | 2    | 15  |
| 18 |  | 25    | 2017 | 21  |
| 19 |  | 2     | 6    | 9   |
| 20 |  | 252   | 24   | 0   |
| 21 |  | 12    | 8    | 10  |
| 22 |  | 26    | 12   | 0   |
| 23 |  | 5814  | 0    | 8   |

[24 rows x 27 columns]

```
[42]: # plot marker genes as dotplot
sc.pl.dotplot(CM_adata, ["MYH6", "MYH7", "SHOX2", "TBX3", "VSNL1", "RSP03", "PRRX1", "NR2F2", "RELN"],
↳ groupby="leiden_1", standard_scale="var", save="Subset_Cardiomyocytes.pdf")
```

WARNING: saving figure to file figures/dotplot\_Subset\_Cardiomyocytes.pdf





```
[43]: CM_adata.obs["CM_clusters"] = CM_adata.obs["leiden_1"].map(
    {
        "0": "vCM",
        "1": "vCM",
        "2": "vCM",
        "3": "vCM",
        "4": "vCM",
        "5": "vCM",
        "6": "vCM",
        "7": "vCM",
        "8": "vCM",
        "9": "vCM",
        "10": "vCM",

```

```

    "11": "vCM",
    "12": "vCM",
    "13": "vCM",
    "14": "aCM",
    "15": "vCM",
    "16": "aCM",
    "17": "vCM",
    "18": "CCS",
    "19": "unknown", # not really strong in any of these CM panel markers
    "20": "aCM",
    "21": "aCM",
    "22": "aCM",
    "23": "vCM"
}
)

```

```
[44]: CM_adata.obs["CM_clusters"][CM_adata.obs["cell_type_mid"] == "AV"] = "CCS"
```

/tmp/ipykernel\_52419/1847081385.py:1: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values`` instead, to perform the assignment in a single step and ensure this keeps updating the original `df``.

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user\\_guide/indexing.html#returning-a-view-versus-a-copy](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
CM_adata.obs["CM_clusters"][CM_adata.obs["cell_type_mid"] == "AV"] = "CCS"
/tmp/ipykernel_52419/1847081385.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user\\_guide/indexing.html#returning-a-view-versus-a-copy](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
CM_adata.obs["CM_clusters"][CM_adata.obs["cell_type_mid"] == "AV"] = "CCS"
```

```
[45]: CM_adata.obs["CM_clusters"][CM_adata.obs["cell_type_mid"] == "ncCM"] = "CCS"
```

/tmp/ipykernel\_52419/2577344311.py:1: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame,

like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values`` instead, to perform the assignment in a single step and ensure this keeps updating the original `df``.

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user\\_guide/indexing.html#returning-a-view-versus-a-copy](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
CM_adata.obs["CM_clusters"][CM_adata.obs["cell_type_mid"] == "ncCM"] = "CCS"
/tmp/ipykernel_52419/2577344311.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user\\_guide/indexing.html#returning-a-view-versus-a-copy](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
CM_adata.obs["CM_clusters"][CM_adata.obs["cell_type_mid"] == "ncCM"] = "CCS"
```

```
[46]: pd.crosstab(CM_adata.obs["CM_clusters"], CM_adata.obs["cell_type_mid"])
```

```
[46]: cell_type_mid  AV  Cardiomyocytes  Immat_CM  Immature_Cardiomyocytes  LA1  \
CM_clusters
CCS             638             134             1                 8  140
aCM              0             2917            135            144  413
unknown          0              4             77             9   0
vCM              0            5097            1300           1040  98

cell_type_mid  LA2  LA3  LA4  LA5  LA6  ...  Mat_aCM  Mat_vCM  MetAct_aCM  \
CM_clusters
CCS             8   37   1    1    0  ...    286     11        34
aCM            238  142  143  186  29  ...   2687     26    2471
unknown         0    0    0    0    1  ...    21     17         2
vCM             17   96   15   7    2  ...   763    6139    553

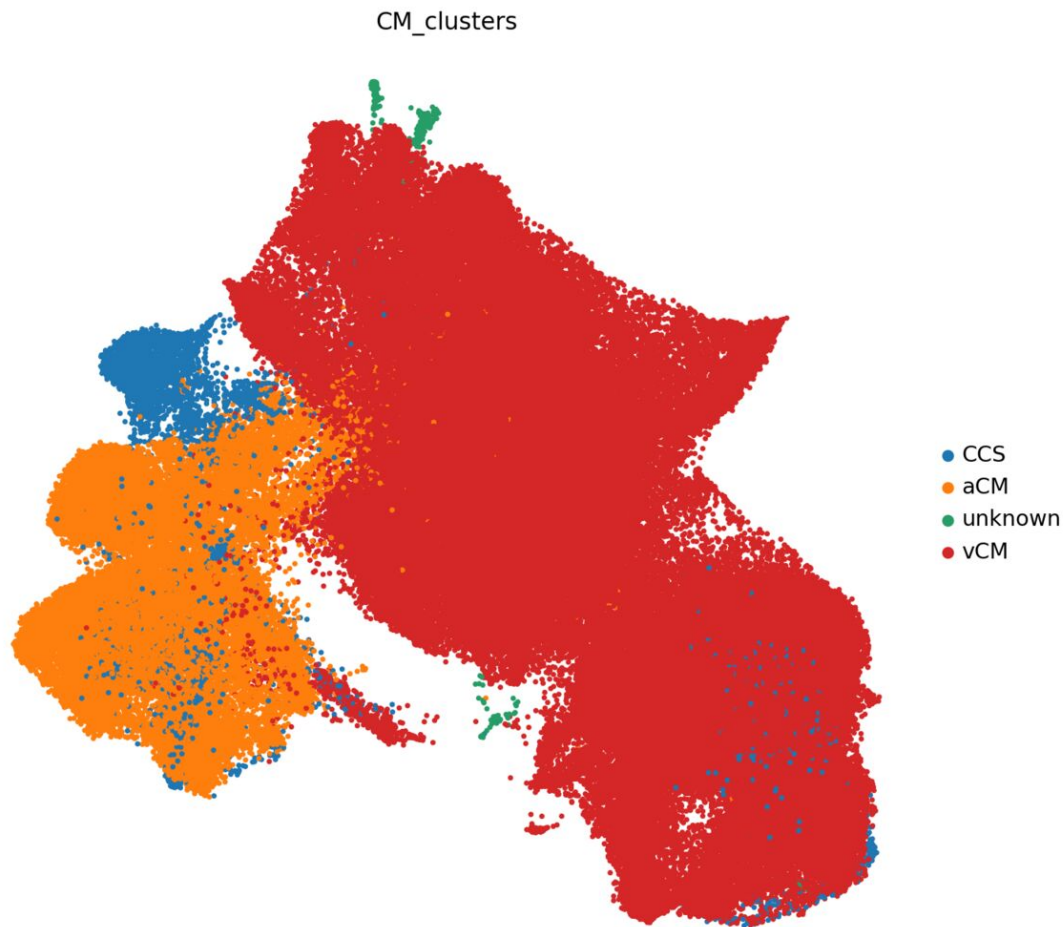
cell_type_mid  MetAct_vCM_1  MetAct_vCM_2  Prol_CM    aCM  \
CM_clusters
CCS                     2             2         1    169
aCM                    375            191        334  18022
unknown                 2             13        123    9
vCM                   2854            1228       1235   244

cell_type_mid  cardiac muscle myoblast  ncCM    vCM
CM_clusters
CCS                     25    2633     21
aCM                     489     0     20
unknown                  2     0     9
vCM                   63952     0  28292

[4 rows x 27 columns]
```

```
[47]: sc.set_figure_params(figsize=(10, 10), frameon=False)
sc.pl.umap(
    CM_adata,
    color=["CM_clusters"],
    ncols=1,
    #legend_loc = "on data",
    s=40,
    save="_Cardiomyocyte_Postclustering.pdf"
)
```

WARNING: saving figure to file figures/umap\_Cardiomyocyte\_Postclustering.pdf



```
[48]: # pull new annotations back to the object
```

```
[49]: adata.obs["new_clusters"] = adata.obs["merged_labels"]
```

```
[50]: CM_adata.obs["CM_clusters"] = CM_adata.obs["CM_clusters"].cat.set_categories(adata.obs["merged_labels"].cat.  
↳categories)
```

```
[51]: adata.obs["new_clusters"][CM_adata.obs_names] = CM_adata.obs["CM_clusters"]
```

/tmp/ipykernel\_52419/863416275.py:1: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

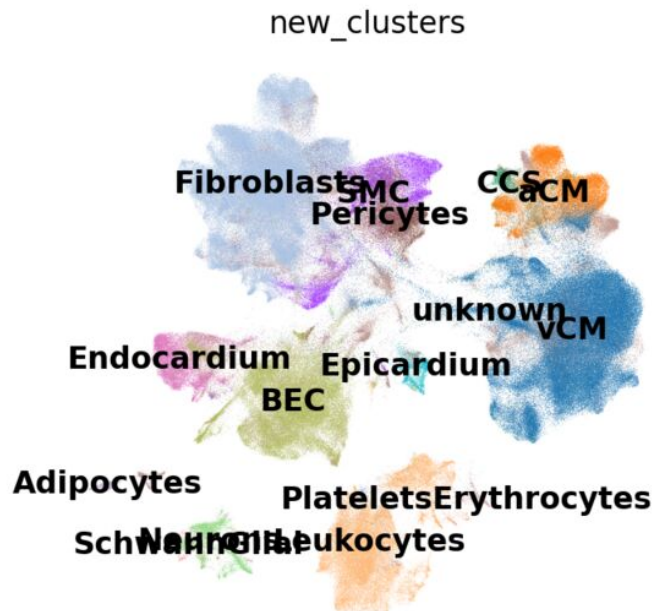
```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values`` instead, to perform the assignment in a single step and ensure this keeps updating the original `df``.

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user\\_guide/indexing.html#returning-a-view-versus-a-copy](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
adata.obs["new_clusters"][CM_adata.obs_names] = CM_adata.obs["CM_clusters"]
```

```
[52]: sc.set_figure_params(figsize=(5, 5), frameon=False)
      sc.pl.umap(
          adata,
          color=["new_clusters"],
          ncols=1,
          legend_loc = "on data"
      )
```



```
[53]: adata.write("data/suspension/anndata_complete.h5ad")
```

### 1.1.3 Endothelial subclustering

```
[54]: # combined dataset now
      #adata = sc.read_h5ad("data/suspension/anndata_combined.h5ad")

[55]: #adata.obs["clusters_res_low"] = bdata.obs["clusters_res_low"]

[56]: # subset object(s) for further processing
      # Cardiomyocytes only
```

```
Endothelialadata = adata[np.isin(adata.obs["merged_labels"], ["BEC"])] .copy()
```

```
[57]: # casting as CSR matrix
Endothelialadata.layers["counts"] = Endothelialadata.layers["counts"].tocsr()
```

```
[58]: sc.pp.highly_variable_genes(
    Endothelialadata,
    n_top_genes=4000,
    subset=True,
    layer="counts",
    flavor="seurat_v3",
    batch_key="dataset"
)
```

```
[59]: Endothelialadata
```

```
[59]: AnnData object with n_obs × n_vars = 57176 × 4000
      obs: 'stage', 'disease_condition', 'cell_or_nuclei', 'n_genes_by_counts',
      'log1p_n_genes_by_counts', 'total_counts', 'log1p_total_counts',
      'pct_counts_in_top_50_genes', 'pct_counts_in_top_100_genes',
      'pct_counts_in_top_200_genes', 'pct_counts_in_top_500_genes', 'total_counts_mt',
      'log1p_total_counts_mt', 'pct_counts_mt', 'total_counts_ribo',
      'log1p_total_counts_ribo', 'pct_counts_ribo', 'total_counts_hb',
      'log1p_total_counts_hb', 'pct_counts_hb', 'dataset', 'disease_subcategory',
      'donor', 'tech', 'cell_type_mid', 'leiden_1.5', 'leiden_1', 'leiden_0.5',
      'leiden_0.2', 'merged_labels', 'new_clusters'
      var: 'highly_variable', 'highly_variable_rank', 'means', 'variances',
      'variances_norm', 'highly_variable_nbatches'
      uns: 'log1p', 'dendrogram_leiden_1.5', 'new_clusters_colors', 'hvg'
      obsm: 'X_scVI', 'X_umap'
      layers: 'counts'
```

```
[60]: variables2merge = ["stage", "cell_or_nuclei", "donor", "tech", "disease_condition"]
```

```
[61]: scvi.model.SCVI.setup_anndata(
    Endothelialadata,
    layer="counts",
    categorical_covariate_keys=variables2merge,
    continuous_covariate_keys=None,
)
```

```
[62]: Endothelial_model = scvi.model.SCVI(
    Endothelialadata,
    n_layers=4,
    n_hidden=128,
    n_latent=20,
    gene_likelihood="nb"
)
```

```
[63]: datasplitter_kwargs = {}
datasplitter_kwargs["drop_dataset_tail"] = True
datasplitter_kwargs["drop_last"] = False
```

```
[64]: Endothelial_model.train(
    max_epochs=40,
    check_val_every_n_epoch=1,
    accelerator="gpu",
    devices=1
    #datasplitter_kwargs=datasplitter_kwargs,
    #strategy="ddp_notebook_find_unused_parameters_true"
)
```

GPU available: True (cuda), used: True  
TPU available: False, using: 0 TPU cores  
HPU available: False, using: 0 HPUs

```

LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0,1]
/home/vincent/miniconda3/envs/scvi-env/lib/python3.13/site-
packages/lightning/pytorch/trainer/connectors/data_connector.py:433: The
'train_dataloader' does not have many workers which may be a bottleneck.
Consider increasing the value of the `num_workers` argument` to `num_workers=15`
in the `DataLoader` to improve performance.
/home/vincent/miniconda3/envs/scvi-env/lib/python3.13/site-
packages/lightning/pytorch/trainer/connectors/data_connector.py:433: The
'val_dataloader' does not have many workers which may be a bottleneck. Consider
increasing the value of the `num_workers` argument` to `num_workers=15` in the
`DataLoader` to improve performance.

Epoch 40/40: 100%|| 40/40 [02:57<00:00, 4.47s/it, v_num=1,
train_loss=1.29e+3]

`Trainer.fit` stopped: `max_epochs=40` reached.

Epoch 40/40: 100%|| 40/40 [02:57<00:00, 4.44s/it, v_num=1,
train_loss=1.29e+3]

```

```

[65]: # save model
Endothelial_model.save("data/suspension/scvi_model_Endothelial", overwrite=True)
#Endothelial_model = scvi.model.SCVI.load("data/suspension/scvi_model_Endothelial",adata=Endothelial_adata)

[66]: # add latent space to object
SCVI_LATENT_KEY = "X_scVI"
latent = Endothelial_model.get_latent_representation()
Endothelial_adata.obsm[SCVI_LATENT_KEY] = latent

[67]: # use scVI latent space for UMAP generation
sc.pp.neighbors(Endothelial_adata, use_rep="X_scVI", n_neighbors = 15)
sc.tl.umap(Endothelial_adata, min_dist=0.3)

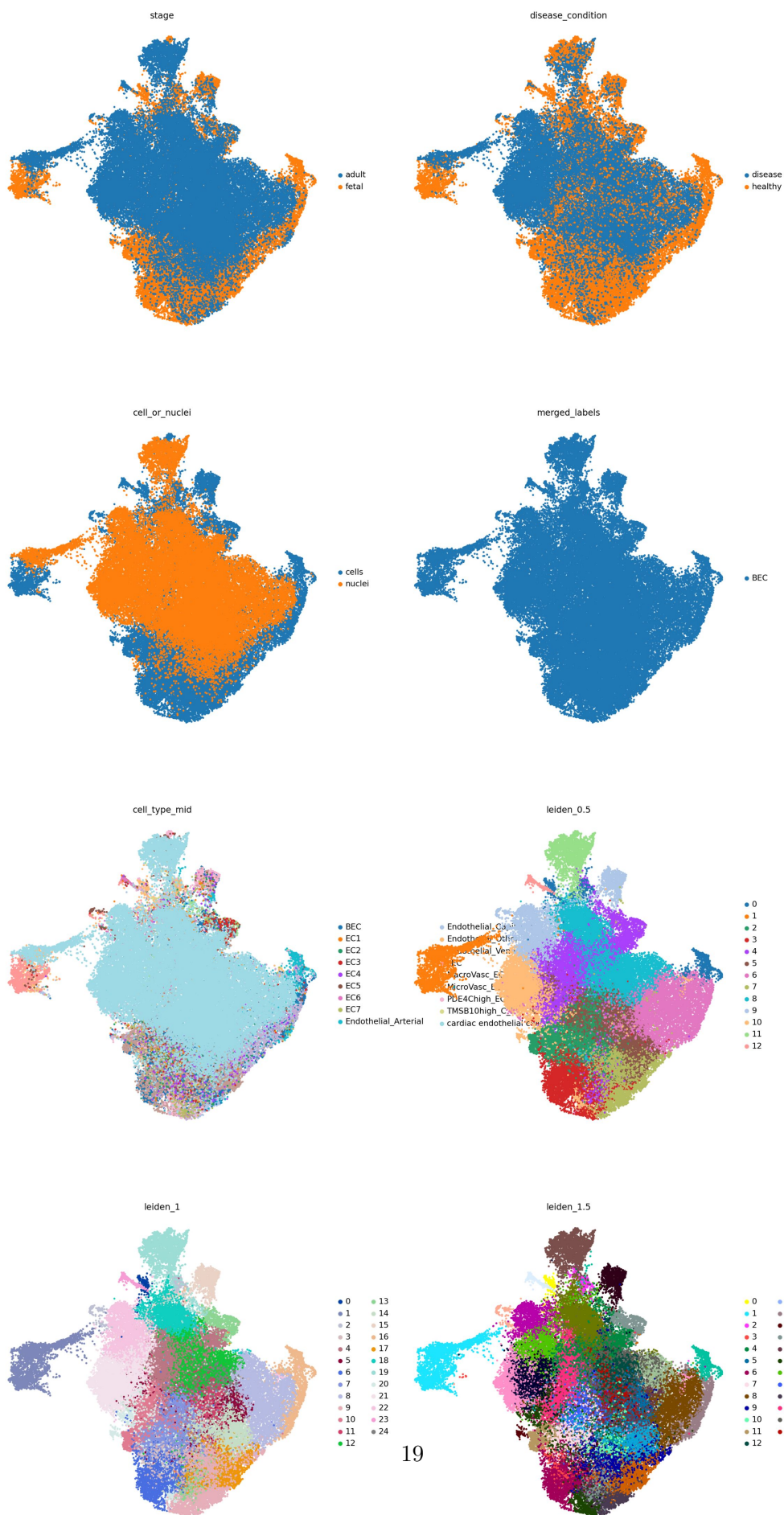
[68]: sc.tl.leiden(Endothelial_adata, flavor="igraph", resolution = 1.5, key_added='leiden_1.5')
sc.tl.leiden(Endothelial_adata, flavor="igraph", resolution = 1, key_added='leiden_1')
sc.tl.leiden(Endothelial_adata, flavor="igraph", resolution = 0.2, key_added='leiden_0.2')
sc.tl.leiden(Endothelial_adata, flavor="igraph", resolution = 0.5, key_added='leiden_0.5')

[69]: sc.set_figure_params(figsize=(10, 10), frameon=False)
sc.pl.umap(
    Endothelial_adata,
    color=["stage", "disease_condition", "cell_or_nuclei", "merged_labels", "cell_type_mid", "leiden_0.5",
↪ "leiden_1", "leiden_1.5"],
    ncols=2,
    #legend_loc = "on data",
    s=40,
    save="_Endothelial_clustering.pdf"
)

```

WARNING: saving figure to file figures/umap\_Endothelial\_clustering.pdf







```
[70]: pd.crosstab(Endothelial_adata.obs["leiden_0.5"], Endothelial_adata.obs["stage"])
```

```
[70]: stage      adult  fetal
leiden_0.5
0           73   1057
1          1283   1033
2          2283   2616
3           947   3021
4          5484   1290
5          5086   1489
6          5154   1910
7          2312   1938
8          8241    934
9          3138   1097
10         4032    501
11         1842    165
12          133    117
```

```
[71]: pd.crosstab(Endothelial_adata.obs["leiden_0.5"], Endothelial_adata.obs["cell_type_mid"]).
↳to_csv("Endothelial_subset_leidenOp5_reannotationMatrix.csv")
pd.crosstab(Endothelial_adata.obs["leiden_0.5"], Endothelial_adata.obs["cell_type_mid"])
```

```
[71]: cell_type_mid  BEC  EC1  EC2  EC3  EC4  EC5  EC6  EC7  Endothelial_Arterial  \
leiden_0.5
0           26    2    0    2    2    5    3   10                436
1            1    0    2    6    1   24    2    1                0
2          1144   26   81   24   68   43    6   51                5
3          1204   25   56    6   95   27    4   79               27
4           554   28   18  253   11   72   11   37               10
5           138    2   32    3   25    5    1    2                1
6           600   21   61   23   54   42    8   42              713
7           803   11   55   15  143   21    1   74               47
8           155  233  222   31   89   72  258   76               29
9           290   44   31   11   36   99  112   30               97
10            42   17    8   14    3   14   12   61               12
11             6   10    4    8    0   66    3    7                2
12             2   29    8   11    8    6   66    1                0
```

```
cell_type_mid  Endothelial_Capillaries  Endothelial_Other  Endothelial_Venous  \
leiden_0.5
0                26                427                9
1                29                621                3
2               496                227                12
3               791                 3                 4
4               210                20              719
5               396                10                 4
6               254                 2                 7
7              1465                11              17
8               176                21                 3
9               142                53              18
10              195                11              24
11                8                20              14
12                5                 4                 0
```

```
cell_type_mid  LEC  MacroVasc_EC  MicroVasc_EC  PDE4Chigh_EC  TMSB10high_C_2  \
leiden_0.5
0            3            67            5            45            45
1           781            0            2            0            2
2            0           106           908           57           34
3            0           162          1055           74           66
4            5            3           307           84            9
5            0           262          929            3            3
6            1           857           71           34            3
```

|    |    |     |     |     |     |
|----|----|-----|-----|-----|-----|
| 7  | 0  | 257 | 494 | 33  | 106 |
| 8  | 28 | 148 | 352 | 51  | 96  |
| 9  | 37 | 29  | 128 | 454 | 69  |
| 10 | 1  | 11  | 117 | 245 | 6   |
| 11 | 5  | 4   | 8   | 90  | 21  |
| 12 | 21 | 55  | 3   | 15  | 15  |

```

cell_type_mid cardiac endothelial cell
leiden_0.5
0
1
2
3
4
5
6
7
8
9
10
11
12

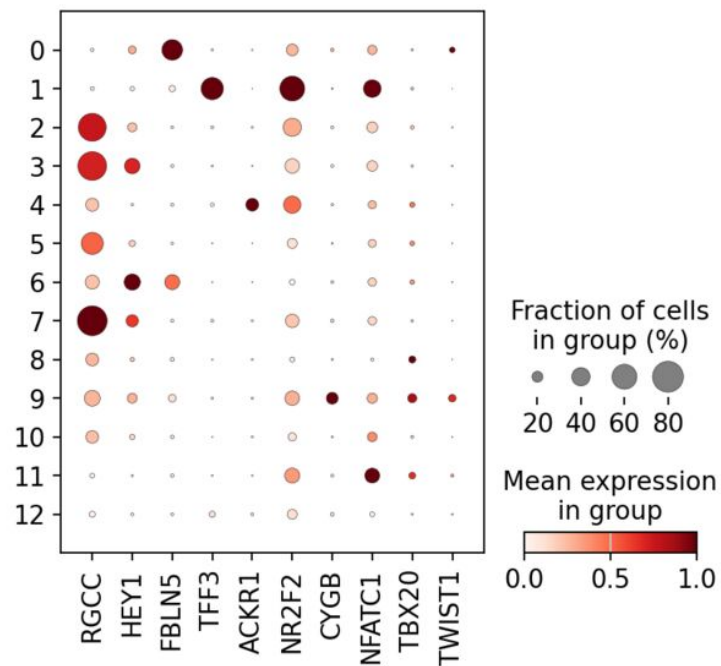
```

```

[72]: # plot marker genes as dotplot
sc.pl.dotplot(Endothelial_adata, ["RGCC", "HEY1", "FBLN5", "TFF3", "ACKR1", "NR2F2", "CYGB", "NFATC1",
↪ "TBX20", "TWIST1"], groupby="leiden_0.5", standard_scale="var", save="Subset_Endothelial.pdf")

```

WARNING: saving figure to file figures/dotplot\_Subset\_Endothelial.pdf



```

[73]: sc.tl.rank_genes_groups(Endothelial_adata, groupby="leiden_0.5", method="wilcoxon")

```

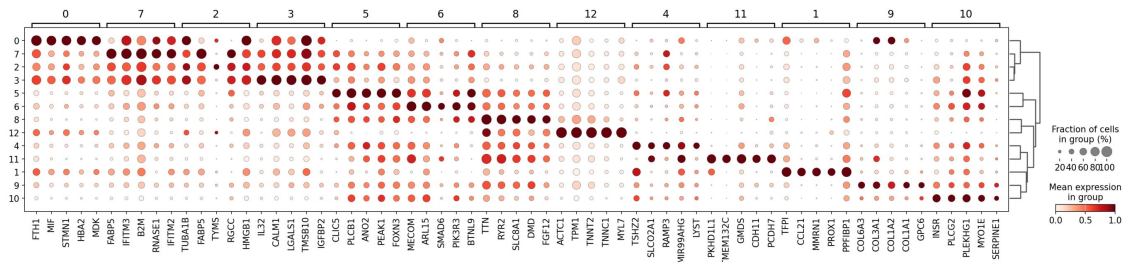
```
[74]: sc.pl.rank_genes_groups_dotplot(Endothelial_adata, groupby="leiden_0.5", standard_scale="var", n_genes=5)
```

WARNING: dendrogram data not found (using key=dendrogram\_leiden\_0.5). Running `sc.tl.dendrogram` with default parameters. For fine tuning it is recommended to run `sc.tl.dendrogram` independently.

/home/vincent/miniconda3/envs/scvi-env/lib/python3.13/site-packages/scanpy/tools/\_dendrogram.py:138: UserWarning: You're trying to run this on 4000 dimensions of `.X`, if you really want this, set `use\_rep='X'`.

Falling back to preprocessing with `sc.pp.pca` and default params.

\_choose\_representation(adata, use\_rep=use\_rep, n\_pcs=n\_pcs)



```
[75]: Endothelial_adata.obs["Endothelial_clusters"] = Endothelial_adata.obs["leiden_0.5"].map(
    {
        "0": "EC_Arterial",
        "1": "EC_Lymphatic",
        "2": "EC_Capillary",
        "3": "EC_Capillary",
        "4": "EC_Venous",
        "5": "EC_Capillary",
        "6": "EC_Arterial",
        "7": "EC_Capillary",
        "8": "EC_Mixed",
        "9": "EC_Valves_Pericytes", # GPC6, quite fibroblasty
        "10": "EC_Mixed", # diffuse expression
        "11": "EC_Valves_Pericytes", # CDH11 # GPC6
        "12": "unknown" # has quite specific CM markers... unusual and cut out of the analysis
    }
)
```

```
[76]: pd.crosstab(Endothelial_adata.obs["Endothelial_clusters"], Endothelial_adata.obs["cell_type_mid"])
```

```
[76]: cell_type_mid      BEC  EC1  EC2  EC3  EC4  EC5  EC6  EC7  \
Endothelial_clusters
EC_Arterial           626   23   61   25   56   47   11   52
EC_Capillary        3289   64  224   48  331   96   12  206
EC_Lymphatic           1    0    2    6    1   24    2    1
EC_Mixed            197  250  230   45   92   86  270  137
EC_Valves_Pericytes  296   54   35   19   36  165  115   37
EC_Venous            554   28   18  253   11   72   11   37
unknown              2    29    8   11    8    6   66    1

cell_type_mid      Endothelial_Arterial  Endothelial_Capillaries  \
Endothelial_clusters
EC_Arterial                          1149                      280
EC_Capillary                          80                       3148
EC_Lymphatic                           0                        29
EC_Mixed                              41                       371
```

|                     |    |     |
|---------------------|----|-----|
| EC_Valves_Pericytes | 99 | 150 |
| EC_Venous           | 10 | 210 |
| unknown             | 0  | 5   |

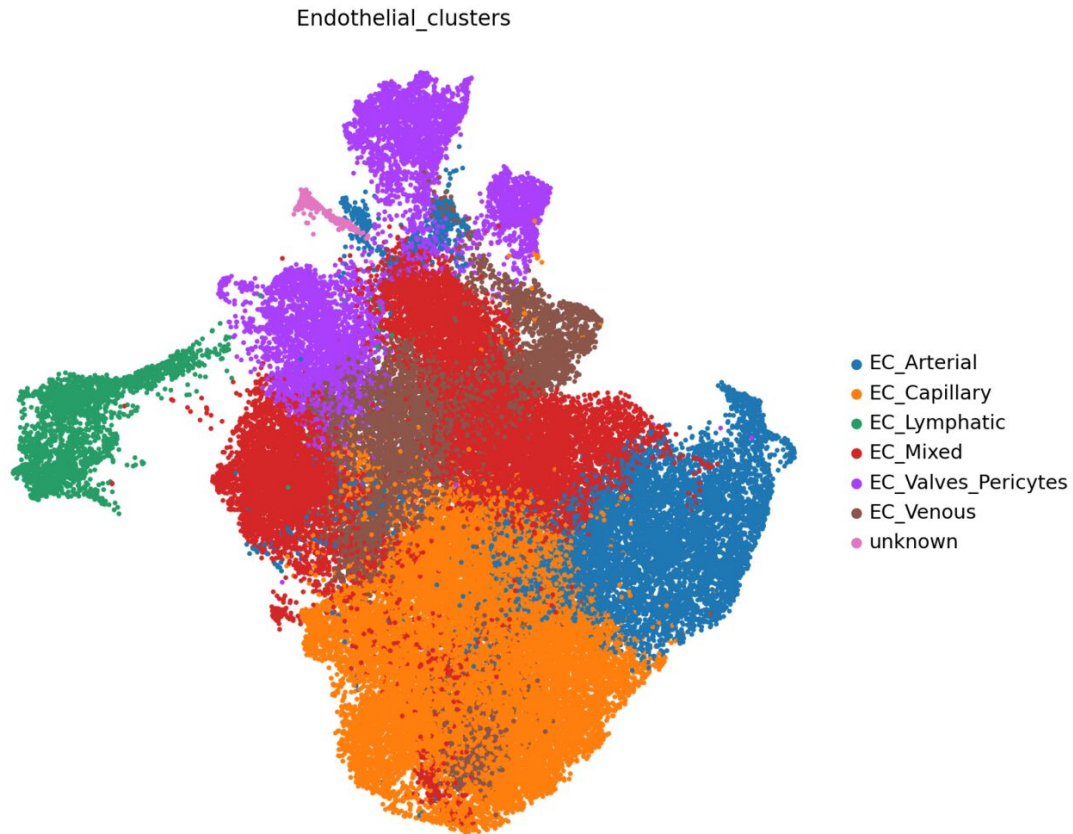
|                      |                   |                    |     |   |
|----------------------|-------------------|--------------------|-----|---|
| cell_type_mid        | Endothelial_Other | Endothelial_Venous | LEC | \ |
| Endothelial_clusters |                   |                    |     |   |
| EC_Arterial          | 429               | 16                 | 4   |   |
| EC_Capillary         | 251               | 37                 | 0   |   |
| EC_Lymphatic         | 621               | 3                  | 781 |   |
| EC_Mixed             | 32                | 27                 | 29  |   |
| EC_Valves_Pericytes  | 73                | 32                 | 42  |   |
| EC_Venous            | 20                | 719                | 5   |   |
| unknown              | 4                 | 0                  | 21  |   |

|                      |              |              |              |   |
|----------------------|--------------|--------------|--------------|---|
| cell_type_mid        | MacroVasc_EC | MicroVasc_EC | PDE4Chigh_EC | \ |
| Endothelial_clusters |              |              |              |   |
| EC_Arterial          | 924          | 76           | 79           |   |
| EC_Capillary         | 787          | 3386         | 167          |   |
| EC_Lymphatic         | 0            | 2            | 0            |   |
| EC_Mixed             | 159          | 469          | 296          |   |
| EC_Valves_Pericytes  | 33           | 136          | 544          |   |
| EC_Venous            | 3            | 307          | 84           |   |
| unknown              | 55           | 3            | 15           |   |

|                      |                |                          |
|----------------------|----------------|--------------------------|
| cell_type_mid        | TMSB10high_C_2 | cardiac endothelial cell |
| Endothelial_clusters |                |                          |
| EC_Arterial          | 48             | 4288                     |
| EC_Capillary         | 209            | 7357                     |
| EC_Lymphatic         | 2              | 841                      |
| EC_Mixed             | 102            | 10875                    |
| EC_Valves_Pericytes  | 90             | 4286                     |
| EC_Venous            | 9              | 4423                     |
| unknown              | 15             | 1                        |

```
[77]: sc.set_figure_params(figsize=(10, 10), frameon=False)
sc.pl.umap(
    Endothelial_adata,
    color=["Endothelial_clusters"],
    ncols=1,
    #legend_loc = "on data",
    s=40,
    save="_Endothelial_Postclustering.pdf"
)
```

WARNING: saving figure to file figures/umap\_Endothelial\_Postclustering.pdf



```
[78]: # pull new annotations back to the object
```

```
[79]: # re_categorise
adata.obs["new_clusters"] = adata.obs["new_clusters"].cat.add_categories(["EC_Mixed", "EC_Venous",
↪ "EC_Arterial", "EC_Capillary", "EC_Lymphatic", "EC_Valves_Pericytes"])
```

```
[80]: Endothelial_adata.obs["Endothelial_clusters"] = Endothelial_adata.obs["Endothelial_clusters"].cat.
↪ set_categories(adata.obs["new_clusters"].cat.categories)
```

```
[81]: adata.obs["new_clusters"][Endothelial_adata.obs_names] = Endothelial_adata.obs["Endothelial_clusters"]
```

/tmp/ipykernel\_52419/534124284.py:1: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values`` instead, to perform the assignment in a single step and ensure this keeps updating the original `df``.

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user\\_guide/indexing.html#returning-a-view-versus-a-copy](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

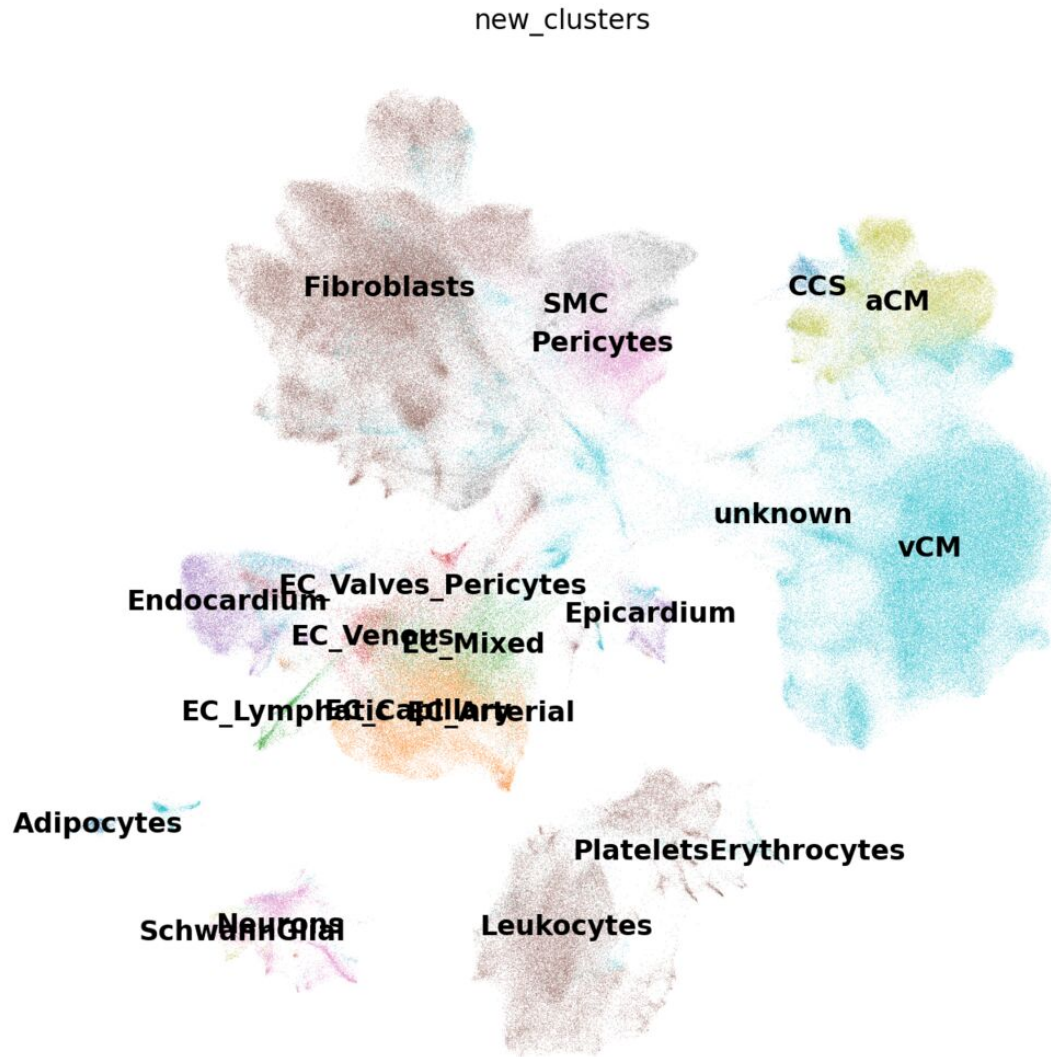
```
adata.obs["new_clusters"][Endothelial_adata.obs_names] =  
Endothelial_adata.obs["Endothelial_clusters"]
```

```
[82]: pd.DataFrame(adata.obs["new_clusters"]).value_counts()
```

```
[82]: new_clusters  
Fibroblasts          130952  
vCM                  114979  
unknown              59263  
Leukocytes           43737  
aCM                  29505  
SMC                  23918  
EC_Capillary         19692  
Pericytes            19296  
Endocardium          15238  
EC_Mixed             13708  
EC_Arterial           8194  
Neurons              7707  
EC_Venous             6774  
EC_Valves_Pericytes  6242  
CCS                   4187  
Epicardium           3430  
EC_Lymphatic          2316  
Adipocytes           1209  
SchwannGlial          683  
PlateletsErythrocytes 611  
otherCM               0  
BEC                   0  
Name: count, dtype: int64
```

```
[83]: # remove old categorise  
adata.obs["new_clusters"] = adata.obs["new_clusters"].cat.remove_categories(  
    ["BEC",  
     "otherCM"  
    ]  
)
```

```
[84]: sc.set_figure_params(figsize=(10, 10), frameon=False)  
sc.pl.umap(  
    adata,  
    color=["new_clusters"],  
    ncols=1,  
    legend_loc = "on data",  
    palette = "tab10"  
)
```



```
[85]:adata.write("data/suspension/anndata_complete.h5ad")
```

```
[ ]:
```

#### 1.1.4 Neuronal subclustering

```
[86]:# combined dataset now
#adata = sc.read_h5ad("data/suspension/anndata_combined.h5ad")
```

```
[87]:#adata.obs["clusters_res_low"] = bdata.obs["clusters_res_low"]
```

```
[88]:# subset object(s) for further processing
Neuron_adata = adata[np.isin(adata.obs["merged_labels"], ["Neurons", "SchwannGlial"])]
```

```
[89]:# casting as CSR matrix
Neuron_adata.layers["counts"] = Neuron_adata.layers["counts"].tocsr()
```

```
[90]: sc.pp.highly_variable_genes(
    Neuron_adata,
    n_top_genes=4000,
    subset=True,
    layer="counts",
    flavor="seurat_v3",
    batch_key="dataset"
)
```

```
[91]: Neuron_adata
```

```
[91]: AnnData object with n_obs × n_vars = 8390 × 4000
      obs: 'stage', 'disease_condition', 'cell_or_nuclei', 'n_genes_by_counts',
      'log1p_n_genes_by_counts', 'total_counts', 'log1p_total_counts',
      'pct_counts_in_top_50_genes', 'pct_counts_in_top_100_genes',
      'pct_counts_in_top_200_genes', 'pct_counts_in_top_500_genes', 'total_counts_mt',
      'log1p_total_counts_mt', 'pct_counts_mt', 'total_counts_ribo',
      'log1p_total_counts_ribo', 'pct_counts_ribo', 'total_counts_hb',
      'log1p_total_counts_hb', 'pct_counts_hb', 'dataset', 'disease_subcategory',
      'donor', 'tech', 'cell_type_mid', 'leiden_1.5', 'leiden_1', 'leiden_0.5',
      'leiden_0.2', 'merged_labels', 'new_clusters'
      var: 'highly_variable', 'highly_variable_rank', 'means', 'variances',
      'variances_norm', 'highly_variable_nbatches'
      uns: 'log1p', 'dendrogram_leiden_1.5', 'new_clusters_colors', 'hvg'
      obsm: 'X_scVI', 'X_umap'
      layers: 'counts'
```

```
[92]: variables2merge = ["stage", "cell_or_nuclei", "donor", "tech", "disease_condition"]
```

```
[93]: scvi.model.SCVI.setup_anndata(
    Neuron_adata,
    layer="counts",
    categorical_covariate_keys=variables2merge,
    continuous_covariate_keys=None,
)
```

```
[94]: Neuron_model = scvi.model.SCVI(
    Neuron_adata,
    n_layers=4,
    n_hidden=128,
    n_latent=20,
    gene_likelihood="nb"
)
```

```
[95]: dataspplitter_kwargs = {}
      dataspplitter_kwargs["drop_dataset_tail"] = True
      dataspplitter_kwargs["drop_last"] = False
```

```
[96]: Neuron_model.train(
    max_epochs=40,
    check_val_every_n_epoch=1,
    accelerator="gpu",
    devices=1
    #dataspplitter_kwargs=dataspplitter_kwargs,
    #strategy="ddp_notebook_find_unused_parameters_true"
)
```

GPU available: True (cuda), used: True

TPU available: False, using: 0 TPU cores

HPU available: False, using: 0 HPUs

LOCAL\_RANK: 0 - CUDA\_VISIBLE\_DEVICES: [0,1]

/home/vincent/miniconda3/envs/scvi-env/lib/python3.13/site-

packages/lightning/pytorch/trainer/connectors/data\_connector.py:433: The



'train\_dataloader' does not have many workers which may be a bottleneck. Consider increasing the value of the `num\_workers` argument` to `num\_workers=15` in the `DataLoader` to improve performance.

/home/vincent/miniconda3/envs/scvi-env/lib/python3.13/site-packages/lightning/pytorch/trainer/connectors/data\_connector.py:433: The 'val\_dataloader' does not have many workers which may be a bottleneck. Consider increasing the value of the `num\_workers` argument` to `num\_workers=15` in the `DataLoader` to improve performance.

Epoch 40/40: 100%|| 40/40 [00:26<00:00, 1.57it/s, v\_num=1, train\_loss=1.36e+3]

`Trainer.fit` stopped: `max\_epochs=40` reached.

Epoch 40/40: 100%|| 40/40 [00:26<00:00, 1.52it/s, v\_num=1, train\_loss=1.36e+3]

```
[97]: # save model
Neuron_model.save("data/suspension/scvi_model_Neuron", overwrite=True)
#Neuron_model = scvi.model.SCVI.load("data/suspension/scvi_model_Neuron",adata=Neuron_adata)
```

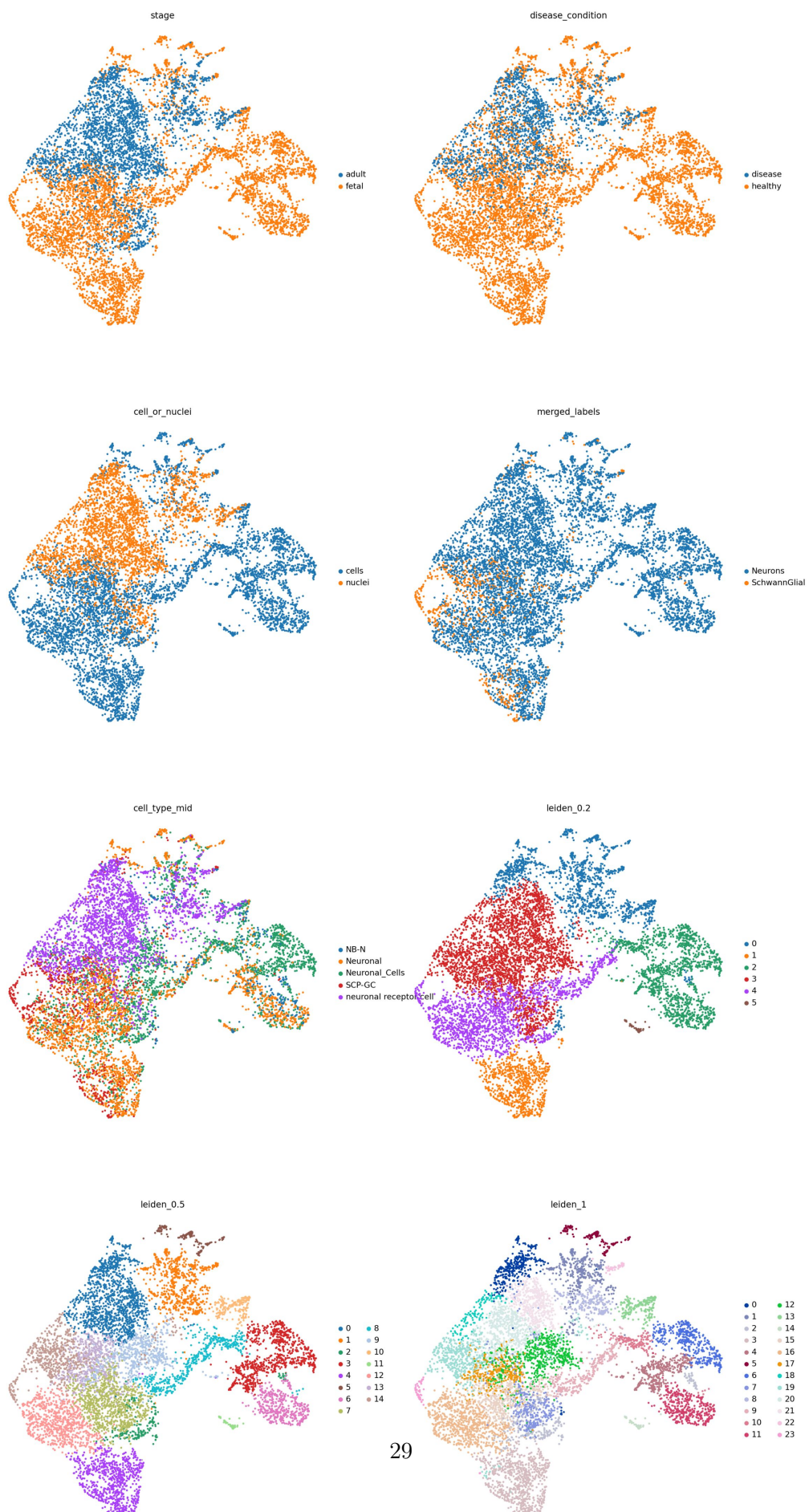
```
[98]: # add latent space to object
SCVI_LATENT_KEY = "X_scVI"
latent = Neuron_model.get_latent_representation()
Neuron_adata.obsm[SCVI_LATENT_KEY] = latent
```

```
[99]: # use scVI latent space for UMAP generation
sc.pp.neighbors(Neuron_adata, use_rep="X_scVI", n_neighbors = 15)
sc.tl.umap(Neuron_adata, min_dist=0.3)
```

```
[100]: sc.tl.leiden(Neuron_adata, flavor="igraph", resolution = 1.5, key_added='leiden_1.5')
sc.tl.leiden(Neuron_adata, flavor="igraph", resolution = 1, key_added='leiden_1')
sc.tl.leiden(Neuron_adata, flavor="igraph", resolution = 0.2, key_added='leiden_0.2')
sc.tl.leiden(Neuron_adata, flavor="igraph", resolution = 0.5, key_added='leiden_0.5')
```

```
[101]: sc.set_figure_params(figsize=(10, 10), frameon=False)
sc.pl.umap(
    Neuron_adata,
    color=["stage", "disease_condition", "cell_or_nuclei", "merged_labels", "cell_type_mid", "leiden_0.2",
↪ "leiden_0.5", "leiden_1"],
    ncols=2,
    #legend_loc = "on data",
    s=40,
    save="_Neurons_clustering.pdf"
)
```

WARNING: saving figure to file figures/umap\_Neurons\_clustering.pdf



```
[102]: pd.crosstab(Neuron_adata.obs["leiden_0.5"], Neuron_adata.obs["stage"])
```

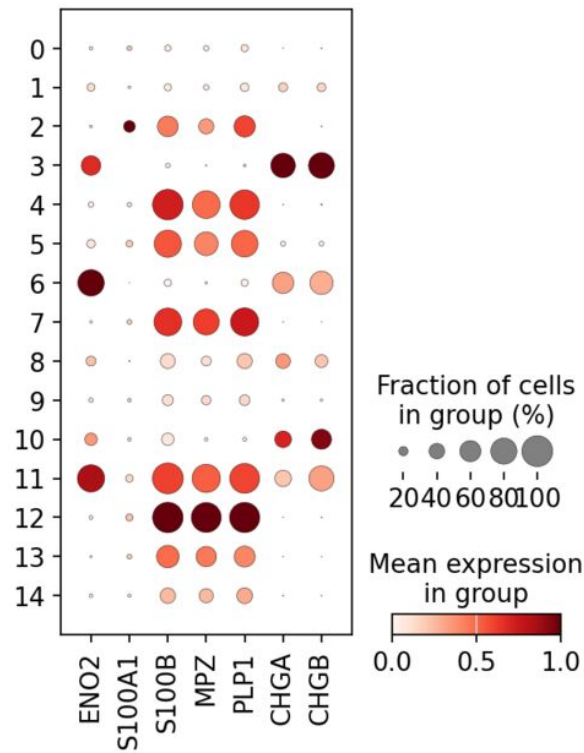
```
[102]: stage      adult  fetal
leiden_0.5
0          1134    124
1           405    264
2           139     52
3            0    782
4            4    774
5            21    131
6            0    432
7           181    722
8            13    462
9           458    113
10           84    114
11            0     46
12           40    727
13          215    341
14          408    204
```

```
[103]: pd.crosstab(Neuron_adata.obs["leiden_0.2"], Neuron_adata.obs["cell_type_mid"]).
↳to_csv("Neuron_subset_leidenOp5_reannotationMatrix.csv")
pd.crosstab(Neuron_adata.obs["leiden_0.2"], Neuron_adata.obs["cell_type_mid"])
```

```
[103]: cell_type_mid NB-N  Neuronal  Neuronal_Cells  SCP-GC  neuronal receptor cell
leiden_0.2
0          132     243          336      66              667
1           7     505          134     131              0
2          124     436          834      1              5
3           17     368          619     234             1703
4           21     816          666     251              28
5           10      22           14      0              0
```

```
[105]: # plot marker genes as dotplot
sc.pl.dotplot(Neuron_adata, ["ENO2", "S100A1", "S100B", "MPZ", "PLP1", "CHGA", "CHGB"], groupby="leiden_0.
↳5", standard_scale="var", save="Subset_Neurons.pdf")
```

WARNING: saving figure to file figures/dotplot\_Subset\_Neurons.pdf



```
[106]: sc.tl.rank_genes_groups(Neuron_adata, groupby="leiden_0.5", method="wilcoxon")
```

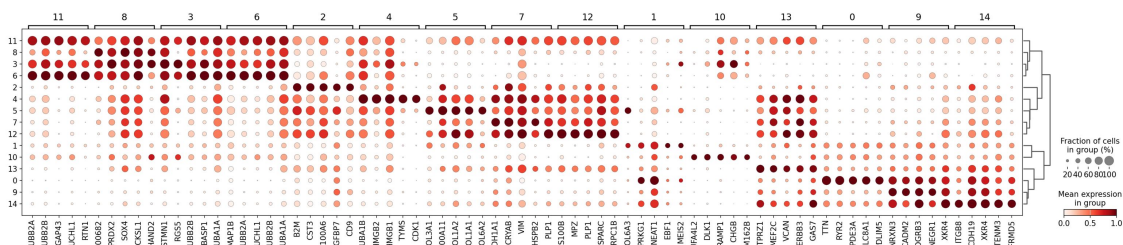
```
[107]: sc.pl.rank_genes_groups_dotplot(Neuron_adata, groupby="leiden_0.5", standard_scale="var", n_genes=5)
```

WARNING: dendrogram data not found (using key=dendrogram\_leiden\_0.5). Running `sc.tl.dendrogram` with default parameters. For fine tuning it is recommended to run `sc.tl.dendrogram` independently.

/home/vincent/miniconda3/envs/scvi-env/lib/python3.13/site-packages/scanpy/tools/\_dendrogram.py:138: UserWarning: You're trying to run this on 4000 dimensions of `.X`, if you really want this, set `use\_rep='X'`.

Falling back to preprocessing with `sc.pp.pca` and default params.

\_choose\_representation(adata, use\_rep=use\_rep, n\_pcs=n\_pcs)



```
[112]: Neuron_adata.obs["Neuron_clusters"] = Neuron_adata.obs["leiden_0.5"].map(
    {
        "0": "Neurons_Mixed", # not sure, some contamination perhaps also
        "1": "Neurons_Mixed", # not sure, some contamination perhaps also
        "2": "Neurons_SchwannGlialCells", # not sure, some contamination perhaps also
        "3": "Neurons_Mixed",
        "4": "Neurons_SchwannGlialCells",
        "5": "Neurons_SchwannGlialCells",
        "6": "Neurons_Mixed",
        "7": "Neurons_SchwannGlialCells",
        "8": "Neurons_Mixed",
        "9": "Neurons_SchwannGlialCells",
        "10": "Neurons_Mixed",
        "11": "Neurons_Mixed",
        "12": "Neurons_SchwannGlialCells",
        "13": "Neurons_SchwannGlialCells",
        "14": "Neurons_SchwannGlialCells"
    }
)
```

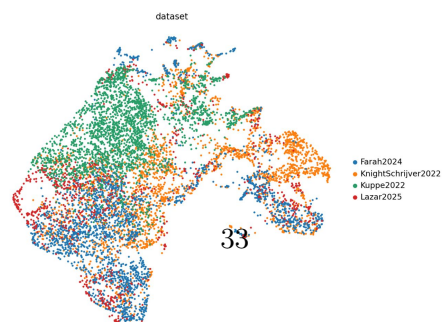
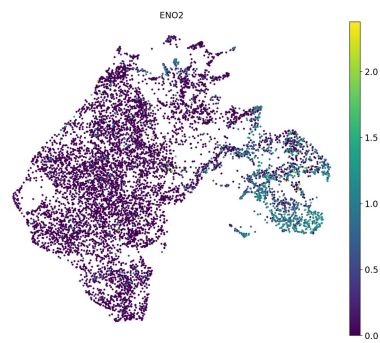
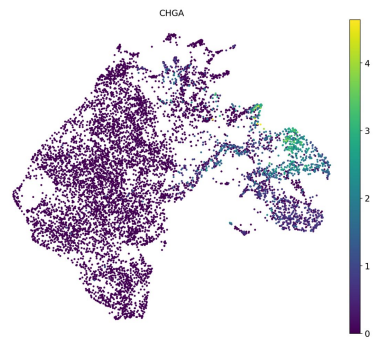
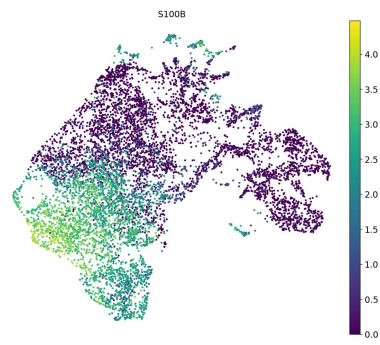
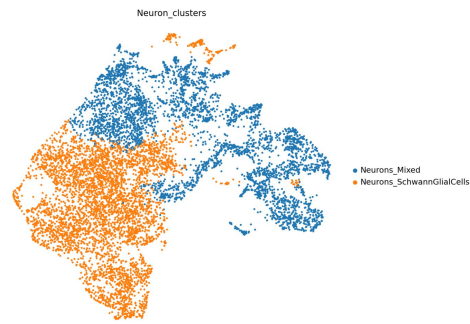
```
[113]: pd.crosstab(Neuron_adata.obs["Neuron_clusters"], Neuron_adata.obs["cell_type_mid"])
```

```
[113]: cell_type_mid      NB-N  Neuronal  Neuronal_Cells  SCP-GC  \
Neuron_clusters
Neurons_Mixed           265      669           1333      69
Neurons_SchwannGlialCells  46     1721           1270     614

cell_type_mid      neuronal receptor cell
Neuron_clusters
Neurons_Mixed              1524
Neurons_SchwannGlialCells      879
```

```
[114]: sc.set_figure_params(figsize=(10, 10), frameon=False)
sc.pl.umap(
    Neuron_adata,
    color=["Neuron_clusters", "S100B", "CHGA", "ENO2", "dataset"],
    ncols=1,
    #legend_loc = "on data",
    s=40,
    save="_Neurons_Postclustering.pdf"
)
```

WARNING: saving figure to file figures/umap\_Neurons\_Postclustering.pdf



```
[115]: # pull new annotations back to the object

[116]: # re_categorise
adata.obs["new_clusters"] = adata.obs["new_clusters"].cat.add_categories(["Neurons_Mixed",
↳ "Neurons_SchwannGlialCells"])

[117]: Neuron_adata.obs["Neuron_clusters"] = Neuron_adata.obs["Neuron_clusters"].cat.set_categories(adata.
↳ obs["new_clusters"].cat.categories)

[118]: adata.obs["new_clusters"][Neuron_adata.obs_names] = Neuron_adata.obs["Neuron_clusters"]
```

/tmp/ipykernel\_52419/157910487.py:1: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!  
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.  
A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user\\_guide/indexing.html#returning-a-view-versus-a-copy](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
adata.obs["new_clusters"][Neuron_adata.obs_names] =
Neuron_adata.obs["Neuron_clusters"]
```

```
[119]: pd.DataFrame(adata.obs["new_clusters"]).value_counts()
```

```
[119]: new_clusters
Fibroblasts          130952
vCM                  114979
unknown              59263
Leukocytes           43737
aCM                  29505
SMC                  23918
EC_Capillary         19692
Pericytes            19296
Endocardium          15238
EC_Mixed             13708
EC_Arterial           8194
EC_Venous             6774
EC_Valves_Pericytes   6242
Neurons_SchwannGlialCells 4530
CCS                   4187
Neurons_Mixed         3860
Epicardium            3430
EC_Lymphatic          2316
Adipocytes            1209
PlateletsErythrocytes    611
```



```

Neurons          0
SchwannGlial     0
Name: count, dtype: int64

```

```

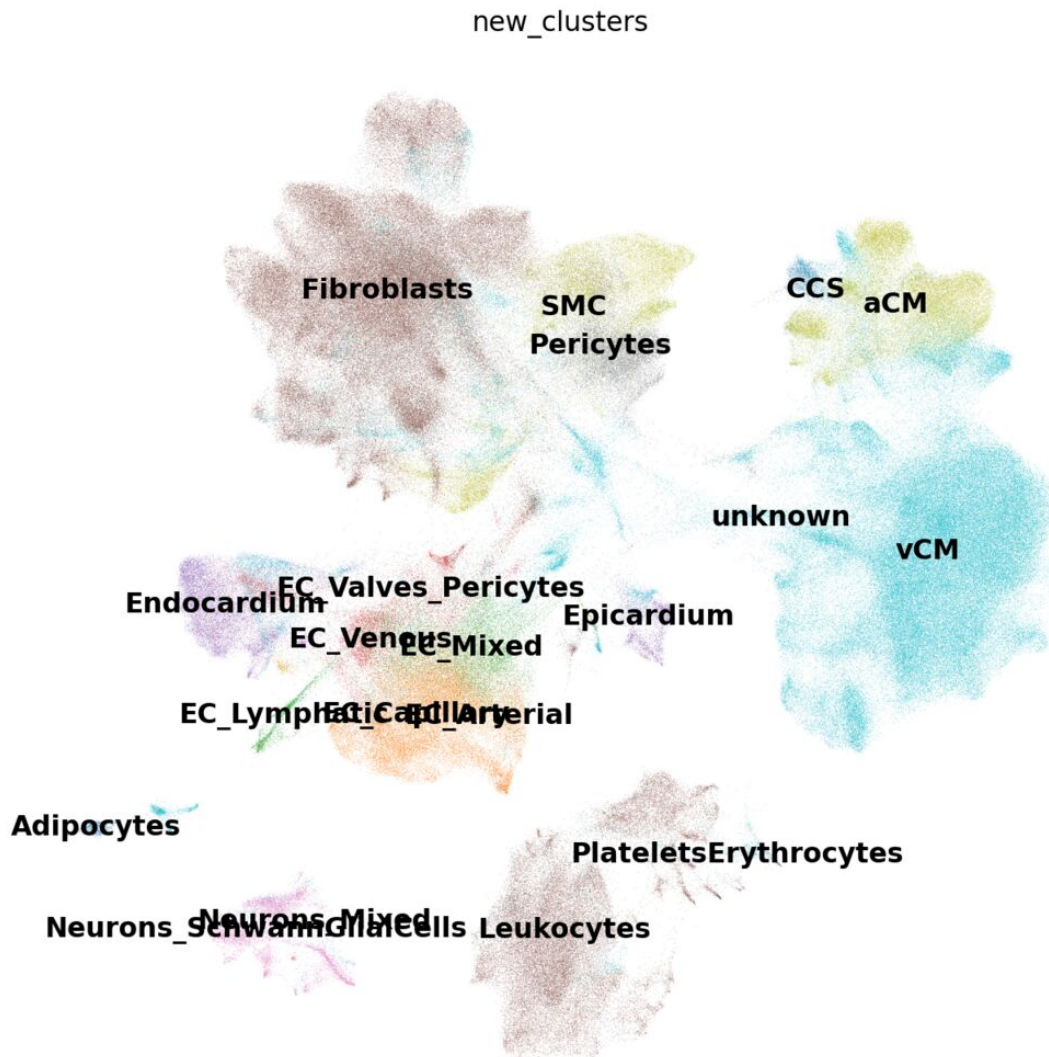
[120]: # remove old categorise
adata.obs["new_clusters"] = adata.obs["new_clusters"].cat.remove_categories(
    ["Neurons",
     "SchwannGlial"]
)

```

```

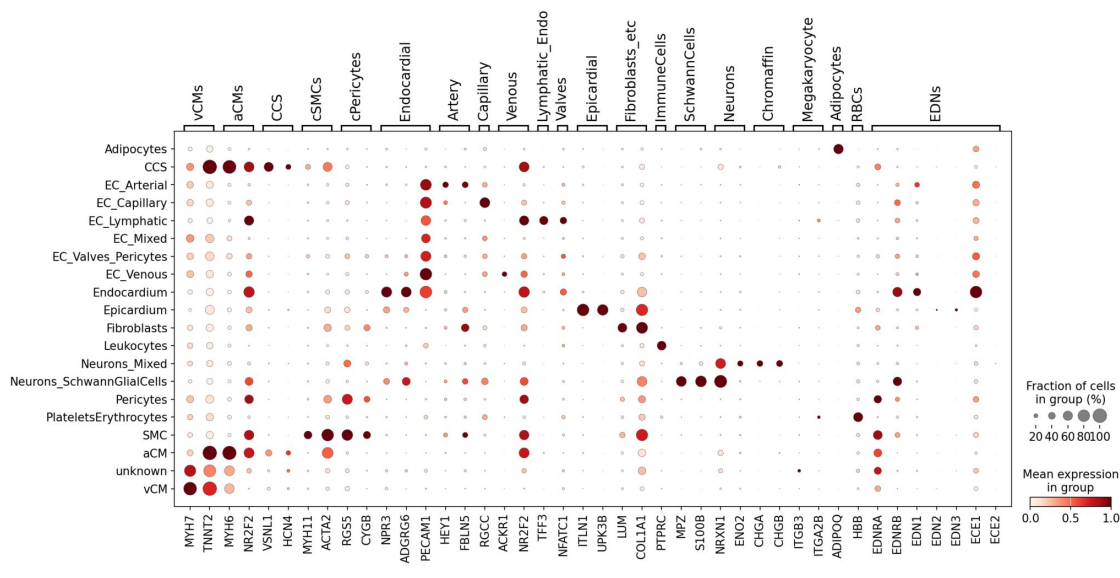
[121]: sc.set_figure_params(figsize=(10, 10), frameon=False)
sc.pl.umap(
    adata,
    color=["new_clusters"],
    ncols=1,
    legend_loc = "on data",
    palette = "tab10"
)

```





```
[122]: # plot marker genes as dotplot
sc.pl.dotplot(adata, marker_genes, groupby="new_clusters", standard_scale="var")
```



```
[123]: adata.write("data/suspension/anndata_complete.h5ad")
```

### 1.1.5 lastly, labelling the DeBono2025 dataset

```
[124]: # combined dataset now
#adata = sc.read_h5ad("data/suspension/anndata_combined.h5ad")
```

```
[125]: #adata.obs["clusters_res_low"] = bdata.obs["clusters_res_low"]
```

```
[126]: # subset object(s) for further processing
debono_adata = adata[np.isin(adata.obs["dataset"], ["DeBono2025"])] .copy()
```

```
[127]: # casting as CSR matrix
debono_adata.layers["counts"] = debono_adata.layers["counts"].tocsr()
```

```
[128]: debono_adata
```

```
[128]: AnnData object with n_obs × n_vars = 49738 × 15092
      obs: 'stage', 'disease_condition', 'cell_or_nuclei', 'n_genes_by_counts',
      'log1p_n_genes_by_counts', 'total_counts', 'log1p_total_counts',
      'pct_counts_in_top_50_genes', 'pct_counts_in_top_100_genes',
      'pct_counts_in_top_200_genes', 'pct_counts_in_top_500_genes', 'total_counts_mt',
      'log1p_total_counts_mt', 'pct_counts_mt', 'total_counts_ribo',
      'log1p_total_counts_ribo', 'pct_counts_ribo', 'total_counts_hb',
      'log1p_total_counts_hb', 'pct_counts_hb', 'dataset', 'disease_subcategory',
      'donor', 'tech', 'cell_type_mid', 'leiden_1.5', 'leiden_1', 'leiden_0.5',
      'leiden_0.2', 'merged_labels', 'new_clusters'
      uns: 'log1p', 'dendrogram_leiden_1.5', 'new_clusters_colors'
      obsm: 'X_scVI', 'X_umap'
      layers: 'counts'
```

```
[129]: debono_adata.obs_names
```

```
[129]: Index(['DeBono2025F8W4D_AAACCCAAGCCTATCA-1',
            'DeBono2025F8W4D_AAACCCAAGGAGGCA-1',
            'DeBono2025F8W4D_AAACCCAAGTTCACGTG-1',
            'DeBono2025F8W4D_AAACCCACAGGTCTCG-1',
            'DeBono2025F8W4D_AAACCCACATAATCGC-1',
            'DeBono2025F8W4D_AAACCCAGTAAGCGGT-1',
            'DeBono2025F8W4D_AAACCCAGTACCCGCA-1',
            'DeBono2025F8W4D_AAACCCAGTGCCTTTC-1',
            'DeBono2025F8W4D_AAACCCAGTGGATGAC-1',
            'DeBono2025F8W4D_AAACCCATCACAGTGT-1',
            ...
            'DeBono2025M10W5D_TTTGTTGGTCCAATCA-1',
            'DeBono2025M10W5D_TTTGTTGGTCCTGGTG-1',
            'DeBono2025M10W5D_TTTGTTGGTGAATTGA-1',
            'DeBono2025M10W5D_TTTGTTGGTTCTCGTC-1',
            'DeBono2025M10W5D_TTTGTTGGTTGCATTG-1',
            'DeBono2025M10W5D_TTTGTTGTCCCGAGAC-1',
            'DeBono2025M10W5D_TTTGTTGTCCCTGGTT-1',
            'DeBono2025M10W5D_TTTGTTGTCTCGACAAG-1',
            'DeBono2025M10W5D_TTTGTTGTCTACCCAC-1',
            'DeBono2025M10W5D_TTTGTTGTCTCCGTGT-1'],
           dtype='object', length=49738)
```

```
[130]: debono_adata.obs["cell_type_mid"].value_counts()
```

```
[130]: cell_type_mid
unlabelled    49738
Name: count, dtype: int64
```

```
[131]: sc.pp.highly_variable_genes(
        debono_adata,
        n_top_genes=5000,
        subset=True,
        layer="counts",
        flavor="seurat_v3",
        batch_key="donor"
    )
```

```
[132]: variables2merge = ["donor"]
```

```
[133]: scvi.model.SCVI.setup_anndata(
        debono_adata,
        layer="counts",
        categorical_covariate_keys=variables2merge,
        continuous_covariate_keys=None,
    )
```

```
[134]: debono_model = scvi.model.SCVI(
        debono_adata,
        n_layers=2,
        n_hidden=128,
        n_latent=40,
        gene_likelihood="nb"
    )
```

```
[135]: datasplitter_kwargs = {}
        datasplitter_kwargs["drop_dataset_tail"] = True
        datasplitter_kwargs["drop_last"] = False
```

```
[136]: debono_model.train(
        max_epochs=40,
        check_val_every_n_epoch=1,
        accelerator="gpu",
        devices=1
        #datasplitter_kwargs=datasplitter_kwargs,
        #strategy="ddp_notebook_find_unused_parameters_true"
```

```
)
```

```
GPU available: True (cuda), used: True
TPU available: False, using: 0 TPU cores
HPU available: False, using: 0 HPUs
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0,1]
/home/vincent/miniconda3/envs/scvi-env/lib/python3.13/site-
packages/lightning/pytorch/trainer/connectors/data_connector.py:433: The
'train_dataloader' does not have many workers which may be a bottleneck.
Consider increasing the value of the `num_workers` argument` to `num_workers=15`
in the `DataLoader` to improve performance.
/home/vincent/miniconda3/envs/scvi-env/lib/python3.13/site-
packages/lightning/pytorch/trainer/connectors/data_connector.py:433: The
'val_dataloader' does not have many workers which may be a bottleneck. Consider
increasing the value of the `num_workers` argument` to `num_workers=15` in the
`DataLoader` to improve performance.

Epoch 40/40: 100%|| 40/40 [02:14<00:00, 3.32s/it, v_num=1,
train_loss=1.88e+3]

`Trainer.fit` stopped: `max_epochs=40` reached.

Epoch 40/40: 100%|| 40/40 [02:14<00:00, 3.37s/it, v_num=1,
train_loss=1.88e+3]
```

```
[137]: # save model
debono_model.save("data/suspension/scvi_model_debono", overwrite=True)
#debono_model = scvi.model.SCVI.load("data/suspension/scvi_model_debono", adata=debono_adata)
```

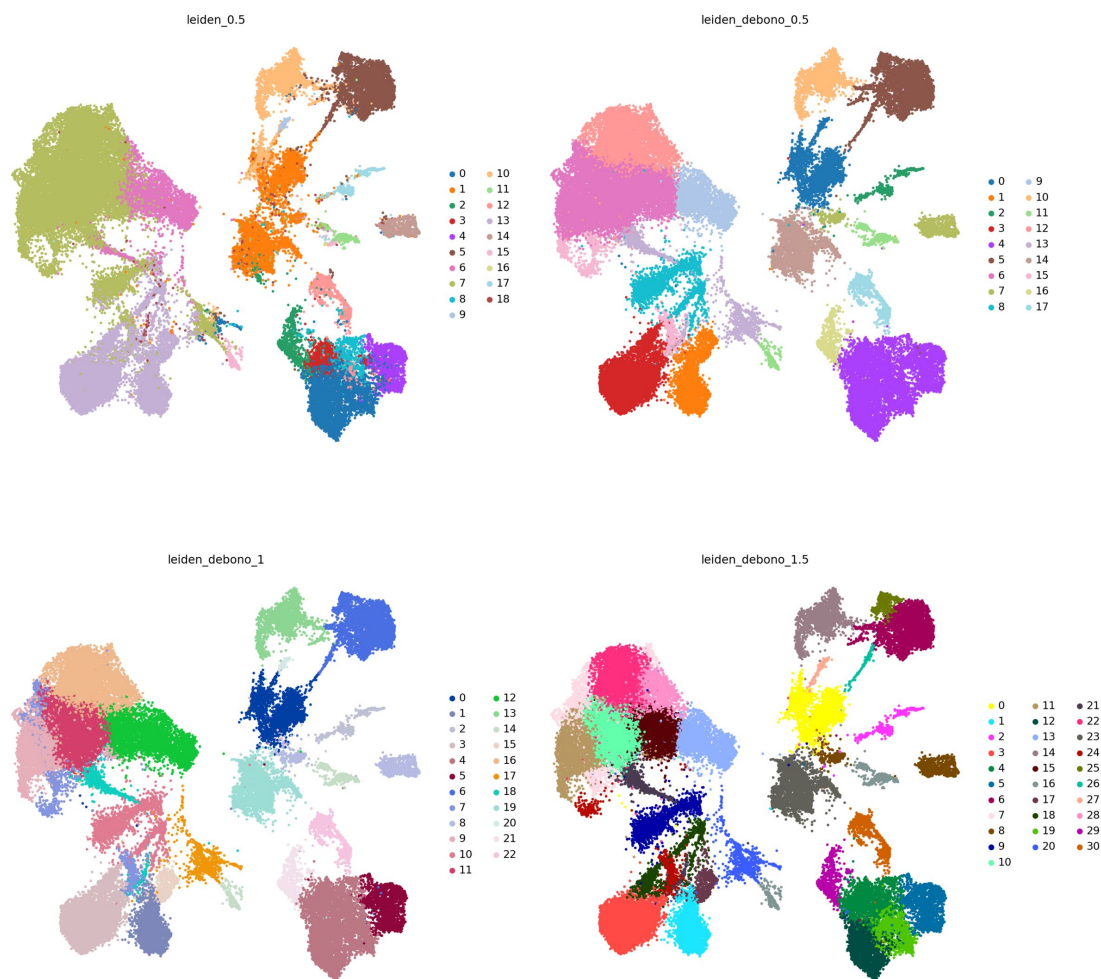
```
[138]: # add latent space to object
SCVI_LATENT_KEY = "X_scVI"
latent = debono_model.get_latent_representation()
debono_adata.obsm[SCVI_LATENT_KEY] = latent
```

```
[139]: # use scVI latent space for UMAP generation
sc.pp.neighbors(debono_adata, use_rep="X_scVI", n_neighbors = 30)
sc.tl.umap(debono_adata, min_dist=0.3)
```

```
[140]: sc.tl.leiden(debono_adata, flavor="igraph", resolution = 1.5, key_added='leiden_debono_1.5')
sc.tl.leiden(debono_adata, flavor="igraph", resolution = 1, key_added='leiden_debono_1')
sc.tl.leiden(debono_adata, flavor="igraph", resolution = 0.2, key_added='leiden_debono_0.2')
sc.tl.leiden(debono_adata, flavor="igraph", resolution = 0.5, key_added='leiden_debono_0.5')
```

```
[141]: sc.set_figure_params(figsize=(10, 10), frameon=False)
sc.pl.umap(
    debono_adata,
    color=["leiden_0.5", "leiden_debono_0.5", "leiden_debono_1", "leiden_debono_1.5"],
    ncols=2,
    #legend_loc = "on data",
    s=40,
    save="_DeBono2025_clustering.pdf"
)
```

WARNING: saving figure to file figures/umap\_DeBono2025\_clustering.pdf



```
[142]: sc.tl.rank_genes_groups(debono_adata, groupby="leiden_debono_1.5", method="wilcoxon")
```

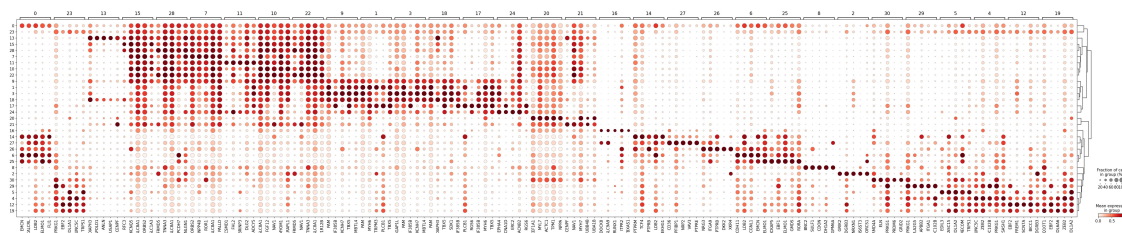
```
[143]: sc.tl.dendrogram(debono_adata, groupby="leiden_debono_1.5")
```

/home/vincent/miniconda3/envs/scvi-env/lib/python3.13/site-packages/scanpy/tools/\_dendrogram.py:138: UserWarning: You're trying to run this on 5000 dimensions of `.X`, if you really want this, set `use\_rep='X'`.

Falling back to preprocessing with `sc.pp.pca` and default params.

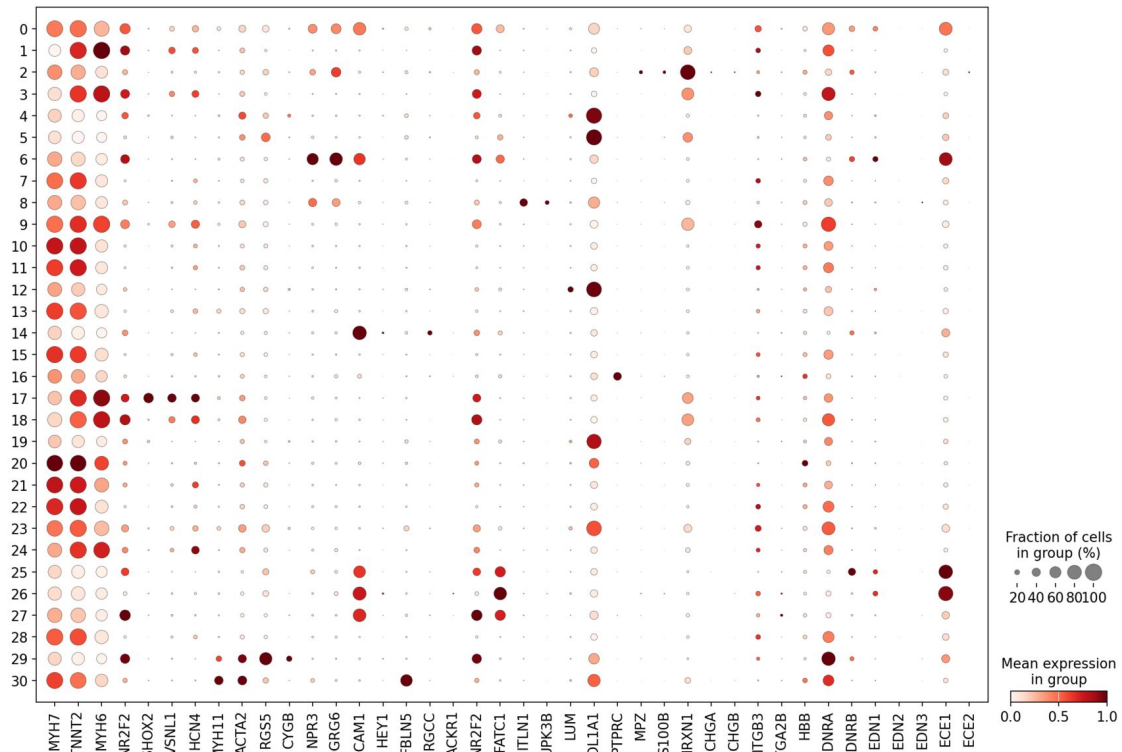
\_choose\_representation(adata, use\_rep=use\_rep, n\_pcs=n\_pcs)

```
[144]: sc.pl.rank_genes_groups_dotplot(debono_adata, groupby="leiden_debono_1.5", standard_scale="var", n_genes=5)
```



```
[146]: # plot marker genes as dotplot
marker_gene_list2 = ["MYH7", "TNNT2", "MYH6", "NR2F2",
"SHOX2", "VSNL1", "HCN4", "MYH11", "ACTA2", "RGS5", "CYGB", "NPR3", "ADGRG6", "PECAM1",
"HEY1", "FBLN5", "RGCC", "ACKR1", "NR2F2", "NFATC1", "ITLN1",
"UPK3B", "LUM", "COL1A1", "PTPRC", "MPZ", "S100B", "NRXN1",
"CHGA", "CHGB", "ITGB3", "ITGA2B", "HBB",
"EDNRA", "EDNRB", "EDN1", "EDN2", "EDN3", "ECE1", "ECE2"]
] # "TFF3" was not found... no lymphatic ECs?

sc.pl.dotplot(debono_adata, marker_gene_list2, groupby="leiden_debono_1.5", standard_scale="var")
```



```
[186]: debono_adata.obs["debono_clusters"] = debono_adata.obs["leiden_debono_1.5"].map(
{
"0": "Endocardium", # quite "doublety", note the MYH and NPR3 expression - common doublets for
↳ sticky CM + Endocardium # Also partially Valve... Valve interstitial, or cushion cells
"1": "aCM",
"2": "Neurons_SchwannGlialCells", # S100B, MPZ, ADGRG6
"3": "aCM", # high TNNT2, MYH6, NR2F2
"4": "Fibroblasts", # Lumican positive
"5": "Fibroblasts", # Messy, but slightly ACTA2+ RGS5+, Lumican negative Fibroblasts?? Pericytes?
"6": "Endocardium",
"7": "vCM",
"8": "Epicardium",
"9": "aCM", # is a mixed population of atrial and ventricular.
"10": "vCM",
"11": "vCM",
```

```

    "12": "Fibroblasts",
    "13": "vCM",
    "14": "EC_Capillary",
    "15": "vCM",
    "16": "Leukocytes",
    "17": "CCS",
    "18": "aCM",
    "19": "Fibroblasts", # this is a mixed population of atrial and ventricular but weak signals -␣
    ↪fibroblast and mixed
    "20": "vCM", # also high HBB content
    "21": "vCM",
    "22": "vCM",
    "23": "vCM", # this is a mixed population of ventricular weak signals
    "24": "CCS", # MYH6 dominant, also HCN4. could be conduction system (AVN)
    "25": "EC_Valves_Pericytes", # PECAM+ NR2F2, and NFATC1 seems valve, cushion endo
    "26": "EC_Valves_Pericytes", # PECAM+ NR2F2, and NFATC1 seems valve, cushion, endo some ACKR1,␣
    ↪suggests venous.. hard to tell
    "27": "EC_Valves_Pericytes", # PECAM+ NR2F2, and NFATC1 seems valve, cushion endo
    "28": "vCM",
    "29": "Pericytes",
    "30": "SMC"
  }
)

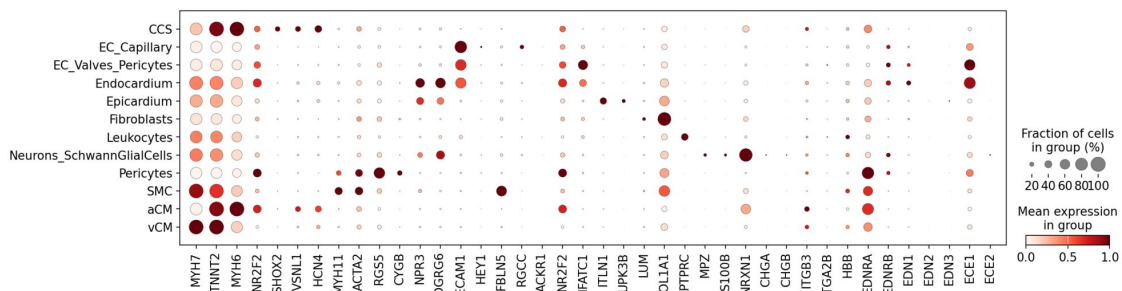
```

```

[187]: # plot marker genes as dotplot
sc.pl.dotplot(debono_adata, marker_gene_list2, groupby="debono_clusters", standard_scale="var",␣
    ↪save="Subset_DeBono2025.pdf")

```

WARNING: saving figure to file figures/dotplot\_Subset\_DeBono2025.pdf



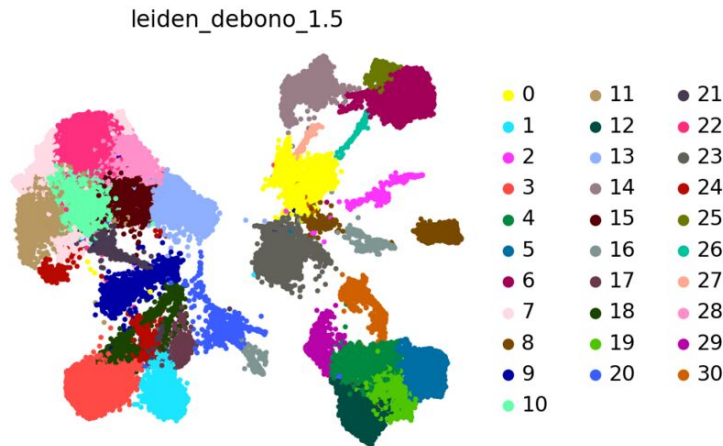
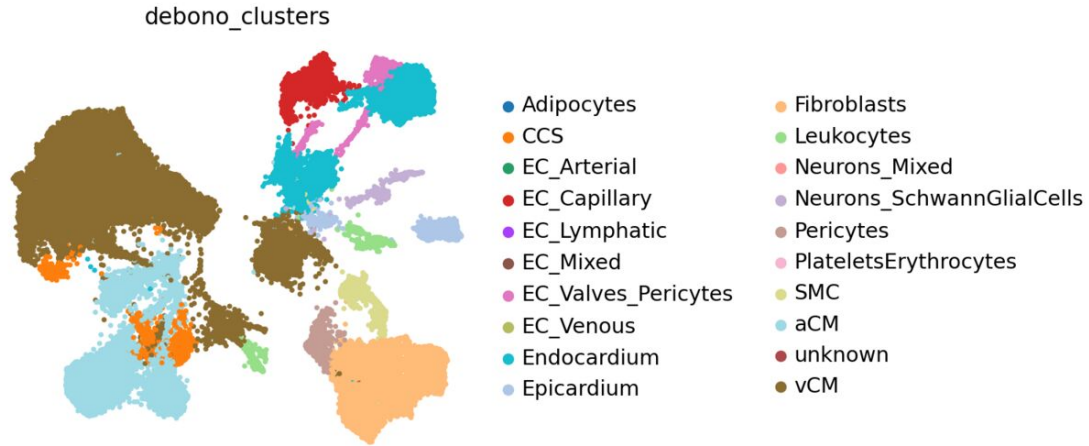
```

[220]: sc.set_figure_params(figsize=(5, 5), frameon=False)
sc.pl.umap(
    debono_adata,
    color=["debono_clusters", "leiden_debono_1.5"],
    ncols=1,
    s=40,
    save="_DeBono2025_Postclustering.pdf"
    #legend_loc = "on data"
)

```

WARNING: saving figure to file figures/umap\_DeBono2025\_Postclustering.pdf





```
[189]: # pull new annotations back to the object
```

```
[190]: debono_adata.obs["debono_clusters"] = debono_adata.obs["debono_clusters"].cat.add_categories(["Adipocytes",
↪ "EC_Venous", "EC_Arterial", "unknown", "EC_Mixed", "Neurons_Mixed", "EC_Lymphatic",
↪ "PlateletsErythrocytes"])
```

```
[191]: adata.obs["new_clusters"].cat.categories
```

```
[191]: Index(['Adipocytes', 'CCS', 'EC_Arterial', 'EC_Capillary', 'EC_Lymphatic',
        'EC_Mixed', 'EC_Valves_Pericytes', 'EC_Venous', 'Endocardium',
        'Epicardium', 'Fibroblasts', 'Leukocytes', 'Neurons_Mixed',
        'Neurons_SchwannGlialCells', 'Pericytes', 'PlateletsErythrocytes',
        'SMC', 'aCM', 'unknown', 'vCM'],
        dtype='object')
```

```
[192]: debono_adata.obs["debono_clusters"].cat.categories
```

```
[192]: Index(['CCS', 'EC_Capillary', 'EC_Valves_Pericytes', 'Endocardium',
        'Epicardium', 'Fibroblasts', 'Leukocytes', 'Neurons_SchwannGlialCells',
        'Pericytes', 'SMC', 'aCM', 'vCM', 'Adipocytes', 'EC_Venous',
        'EC_Arterial', 'unknown', 'EC_Mixed', 'Neurons_Mixed', 'EC_Lymphatic',
        'PlateletsErythrocytes'],
        dtype='object')
```



```
dtype='object')
```

```
[193]: debono_adata.obs["debono_clusters"] = debono_adata.obs["debono_clusters"].cat.reorder_categories(list(adata.  
↳obs["new_clusters"].cat.categories))
```

```
[194]: pd.DataFrame(debono_adata.obs["debono_clusters"].value_counts())
```

```
[194]: debono_clusters  
vCM                22424  
aCM                8603  
Fibroblasts       6555  
Endocardium       5003  
EC_Capillary      1526  
CCS               1137  
Epicardium        1074  
SMC               950  
EC_Valves_Pericytes 726  
Leukocytes        645  
Pericytes         557  
Neurons_SchwannGlialCells 538  
EC_Arterial        0  
Adipocytes         0  
EC_Venous          0  
EC_Lymphatic       0  
EC_Mixed           0  
Neurons_Mixed      0  
PlateletsErythrocytes 0  
unknown           0  
Name: count, dtype: int64
```

```
[195]: adata.obs["new_clusters"][debono_adata.obs_names] = debono_adata.obs["debono_clusters"].copy()
```

/tmp/ipykernel\_52419/2063287939.py:1: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user\\_guide/indexing.html#returning-a-view-versus-a-copy](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
adata.obs["new_clusters"][debono_adata.obs_names] =  
debono_adata.obs["debono_clusters"].copy()
```

```
[196]: pd.DataFrame(adata.obs["new_clusters"].value_counts())
```

```
[196]: new_clusters  
Fibroblasts       137507  
vCM               137403
```

|                           |       |
|---------------------------|-------|
| Leukocytes                | 44382 |
| aCM                       | 38108 |
| SMC                       | 24868 |
| EC_Capillary              | 21218 |
| Endocardium               | 20241 |
| Pericytes                 | 19853 |
| EC_Mixed                  | 13708 |
| unknown                   | 9525  |
| EC_Arterial               | 8194  |
| EC_Valves_Pericytes       | 6968  |
| EC_Venous                 | 6774  |
| CCS                       | 5324  |
| Neurons_SchwannGlialCells | 5068  |
| Epicardium                | 4504  |
| Neurons_Mixed             | 3860  |
| EC_Lymphatic              | 2316  |
| Adipocytes                | 1209  |
| PlateletsErythrocytes     | 611   |

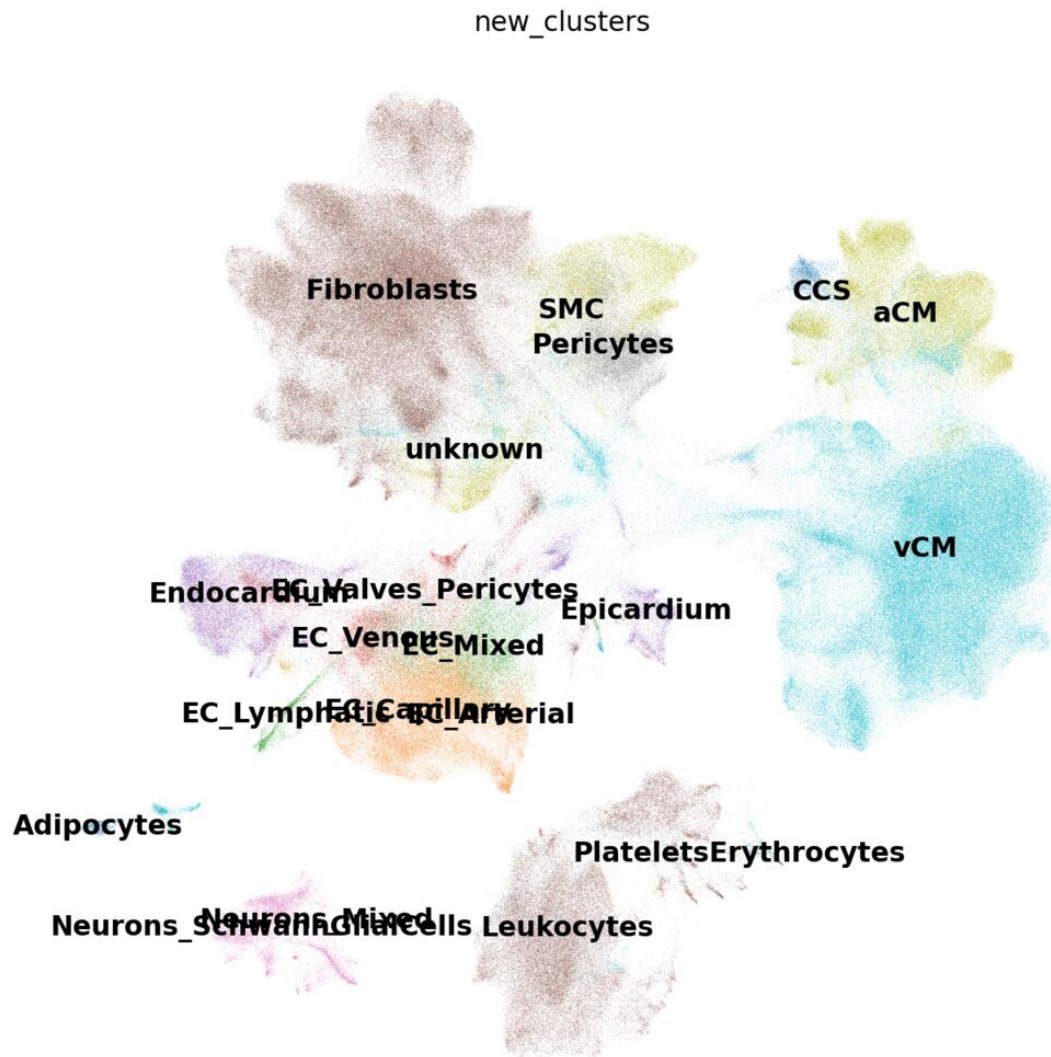
Name: count, dtype: int64

```
[197]: pd.set_option('display.max_rows', 20)
print(pd.DataFrame(adata.obs["cell_type_mid"]).value_counts())
```

| cell_type_mid                |       |
|------------------------------|-------|
| cardiac muscle myoblast      | 64468 |
| Fibro                        | 55823 |
| unlabelled                   | 49738 |
| fibroblast of cardiac tissue | 46901 |
| cardiac endothelial cell     | 32071 |
| ...                          |       |
| MP4                          | 78    |
| MP3                          | 65    |
| LA6                          | 32    |
| FB6                          | 18    |
| P-RBC                        | 17    |

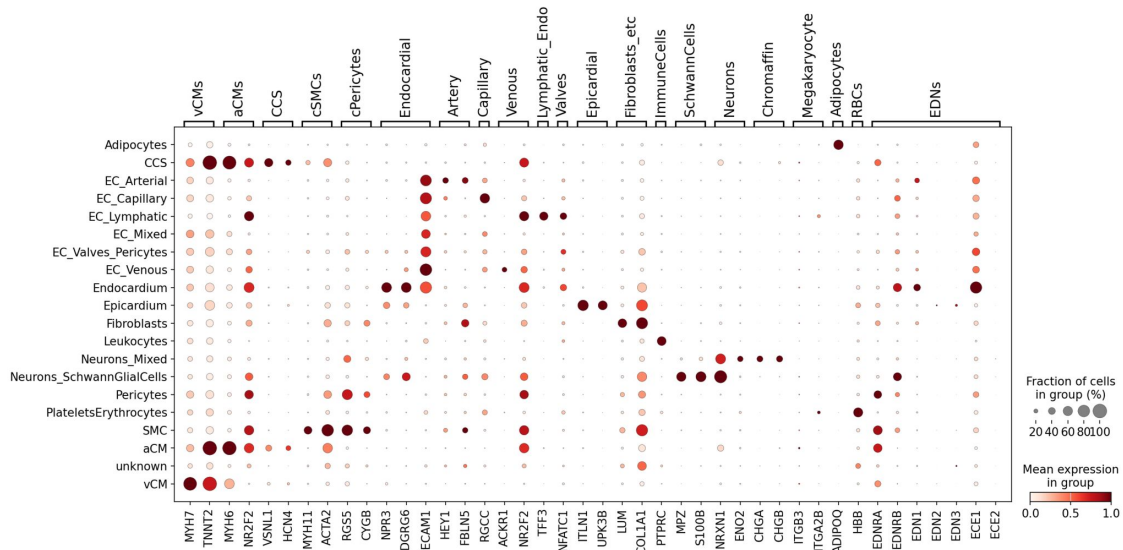
Name: count, Length: 112, dtype: int64

```
[198]: sc.set_figure_params(figsize=(10, 10), frameon=False)
sc.pl.umap(
    adata,
    color=["new_clusters"],
    ncols=1,
    legend_loc = "on data",
    palette = "tab10"
)
```



```
[199]: # plot marker genes as dotplot
sc.pl.dotplot(adata, marker_genes, groupby="new_clusters", standard_scale="var", save="final_NewClusters.
↳pdf")
```

WARNING: saving figure to file figures/dotplot\_final\_NewClusters.pdf



```
[200]:adata.write("data/suspension/anndata_complete.h5ad")
```

```
[212]:marker_gene_list = [
    "TNNT2",
    "MYH7",
    "MYH6", "NR2F2",
    "SHOX2",
    "VSNL1", "HCN4",
    "MYH11", "ACTA2",
    "RGS5", "CYGB",
    "PECAM1",
    "NPR3", "ADGRG6",
    "ACKR1",
    "HEY1", "FBLN5",
    "RGCC",
    "TFF3",
    "NFATC1",
    "ITLN1", "UPK3B",
    "LUM", "COL1A1",
    "NRXN1", "ENO2",
    "CHGA", "CHGB",
    "MPZ", "S100B",
    "PTPRC",
    "ADIPOQ",
    "HBB",
    "ITGB3", "ITGA2B",
    "EDNRB", "EDN1", "EDN2", "EDN3", "ECE1", "ECE2"
]
```

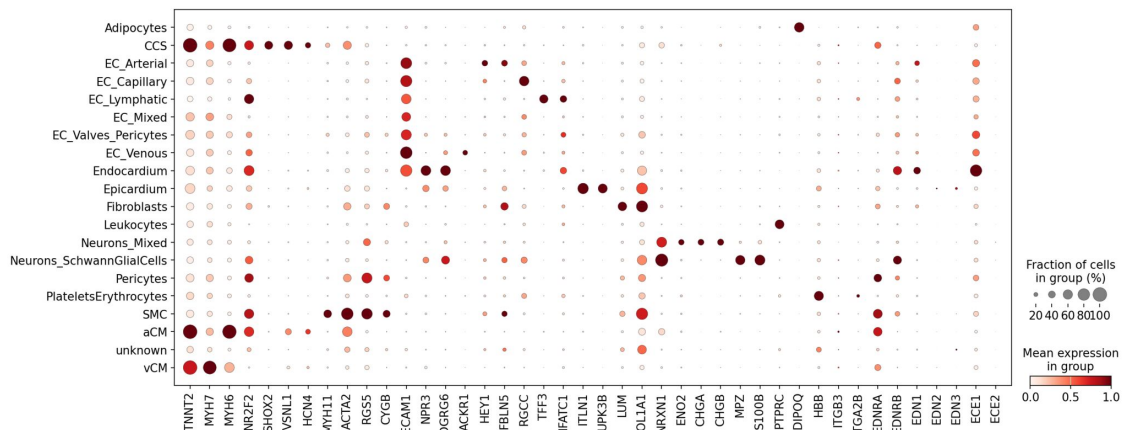
```
[213]:## Write a subset of the dataset...
adata_subset = adata[:,marker_gene_list]
```

```
[214]:del adata_subset.uns
```

```
[215]:adata_subset.write("data/suspension/anndata_markersOnly.h5ad")
```

```
[217]:# plot marker genes as dotplot
sc.pl.dotplot(adata_subset, marker_gene_list, groupby="new_clusters", standard_scale="var",
save="final_NewClusters.pdf")
```

WARNING: saving figure to file figures/dotplot\_final\_NewClusters.pdf



```
[206]: pd.crosstab(adata.obs["new_clusters"], adata.obs["disease_subcategory"])
```

```
[206]: disease_subcategory    CHF    dHF  healthy  myocardial infarction_BZ  \
new_clusters
Adipocytes                  0      0      706                  17
CCS                        29    146     5126                  8
EC_Arterial                51     91     4557                 721
EC_Capillary              176    288    15110                 944
EC_Lymphatic               6      6     1495                  44
EC_Mixed                  127    404     4281                 1262
EC_Valves_Pericytes       59    151     3263                  97
EC_Venous                 48    114     3039                 529
Endocardium                0      0     20241                  0
Epicardium                0      0     4504                  0
Fibroblasts               73    242    99735                 6405
Leukocytes                46    109    21836                 3453
Neurons_Mixed              0      0     2664                  250
Neurons_SchwannGlialCells  0      0     4435                  108
Pericytes                  0      0    10922                 1647
PlateletsErythrocytes      0      0      611                  0
SMC                      140    281    22138                 294
aCM                      183    481    37020                  92
unknown                   9     10     6807                  91
vCM                      438    271    90984                 10493

disease_subcategory  myocardial infarction_FZ  myocardial infarction_IZ  \
new_clusters
Adipocytes                        231                        58
CCS                             11                         0
EC_Arterial                    1423                       153
EC_Capillary                   1997                       134
EC_Lymphatic                    221                       348
EC_Mixed                      2078                       2779
EC_Valves_Pericytes            594                       2147
EC_Venous                     1436                       441
Endocardium                     0                         0
Epicardium                      0                         0
Fibroblasts                    9823                    10017
Leukocytes                     6866                    7347
Neurons_Mixed                   242                       165
Neurons_SchwannGlialCells      160                         5
```

|                       |      |      |
|-----------------------|------|------|
| Pericytes             | 1959 | 1606 |
| PlateletsErythrocytes | 0    | 0    |
| SMC                   | 944  | 510  |
| aCM                   | 141  | 5    |
| unknown               | 130  | 2305 |
| vCM                   | 7437 | 4831 |

|                           |                          |
|---------------------------|--------------------------|
| disease_subcategory       | myocardial infarction_RZ |
| new_clusters              |                          |
| Adipocytes                | 197                      |
| CCS                       | 4                        |
| EC_Arterial               | 1198                     |
| EC_Capillary              | 2569                     |
| EC_Lymphatic              | 196                      |
| EC_Mixed                  | 2777                     |
| EC_Valves_Pericytes       | 657                      |
| EC_Venous                 | 1167                     |
| Endocardium               | 0                        |
| Epicardium                | 0                        |
| Fibroblasts               | 11212                    |
| Leukocytes                | 4725                     |
| Neurons_Mixed             | 539                      |
| Neurons_SchwannGlialCells | 360                      |
| Pericytes                 | 3719                     |
| PlateletsErythrocytes     | 0                        |
| SMC                       | 561                      |
| aCM                       | 186                      |
| unknown                   | 173                      |
| vCM                       | 22949                    |

```
[207]: # Final top markers for supplementary information
```

```
[208]: sc.tl.rank_genes_groups(adata, groupby="new_clusters", method="wilcoxon")
```

```
[211]: results = adata.uns['rank_genes_groups']
groups = results['names'].dtype.names # Get all group names

# Create a single DataFrame with all groups' data
all_degs_df = pd.DataFrame({
    f'{group}_{key}': results[key][group]
    for group in groups
    for key in ['names', 'logfoldchanges', 'pvals', 'pvals_adj']
})

all_degs_df.to_csv("all_DEGs_ranked.csv", index=False)
```

```
[210]: sc.tl.dendrogram(adata, "new_clusters", n_pcs=None, use_rep="X_scVI")
```

```
[219]: sc.pl.rank_genes_groups_dotplot(adata, groupby="new_clusters", standard_scale="var", n_genes=5,
    ↪ save="topDEGs_NewClusters.pdf")
```

WARNING: saving figure to file figures/dotplot\_topDEGs\_NewClusters.pdf

